

Software Engineering

BLOCK 1: Overview of Software Engineering

MCS-213

Block-1

Unit-1 [Software Engineering and its models]

Topics to be covered

Software

Category of Software

What is Software Engineering

Characteristics of software

Software Myths

Qualities Of Software Product

Various Development Models

Capability Maturity Models

Software Process Technology

What is Software

Software is a collection of application programs consisting of:

1. Programs that execute within a computer of any size and architecture.
2. Documents that encompass hard-copy and virtual forms.
3. Data of any form such as numbers, text, video and audio.

Example



Example



Category of Software

- Simple software
 - A small program.
 - designed, developed, used by the same person.
 - no systematic approach is required.
- Industrial strength software
 - use some systematic approach called programming systems.
 - used by different user in different platform
 - complexity is high.

What is Software Engineering

Software engineering is an engineering discipline that is concerned with all aspects of software production.

Software engineering is an engineering discipline whose focus is on cost-effective development with high-quality software systems.

Characteristics Of Software

1. Software is developed - it is not manufacture.
2. Software is highly workable.
3. Software does not "wear out".
4. Most software is created from scratch and not assembled from existing component.

Software Myths

1. Software is easy to change.
2. Computer provide greater reliability than the devices they replace.
3. Testing software or 'providing software correct can remove all the errors.
4. Reusing software increase safety.
5. Software with more feature is better software.

Reasons Of Software Process Fail

1. Not enough time
2. Lack of knowledge
3. Wrong Motivations
4. Insufficient commitment

Qualities Of Software Product

- a) **Correctness:** The software which we are making should meet all the specifications stated by the customer.
- b) **Usability/Learnability:** The amount of efforts or time required to learn how to use the software should be less. This makes the software user-friendly even for IT-illiterate people.
- c) **Integrity :** Just like medicines have side-effects, in the same way a software may have a side-effect i.e. it may affect the working of another application. But a quality software should not have side effects.

Qualities Of Software Product

- d) **Reliability** : The software product should not have any defects. Not only this, it shouldn't fail while execution.
- e) **Efficiency** : This characteristic relates to the way software uses the available resources. The software should make effective use of the storage space and execute command as per desired timing requirements.

Qualities Of Software Product

f) **Security** : With the increase in security threats nowadays, this factor is gaining importance. The software shouldn't have ill effects on data / hardware. Proper measures should be taken to keep data secure from external threats.

g) **Safety** : The software should not be risky to the environment/life.

Software Development

Software is different from other engineering products in the following ways:

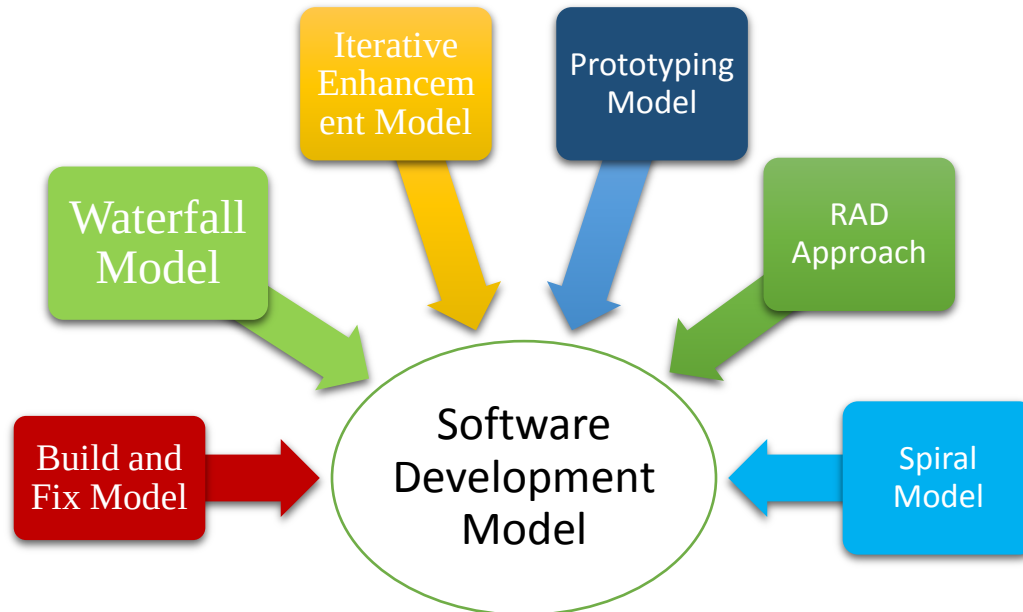
1. Engineering products once developed cannot be changed. To apply modifications the product, redesigning and remanufacturing is required. In the case of software, ultimately changes are to be done in code for any changes to take effect.
2. Unlike most of the other engineering products, software can be reused. Once a piece of code is written for some application, it can be reused.

Software Development

3. The management of software development projects is a highly demanding task. The estimation regarding the time and cost of software needs standardization of developers creativity, which can be a variable quantity. It means that software projects cannot be managed like engineering products.

Development Models

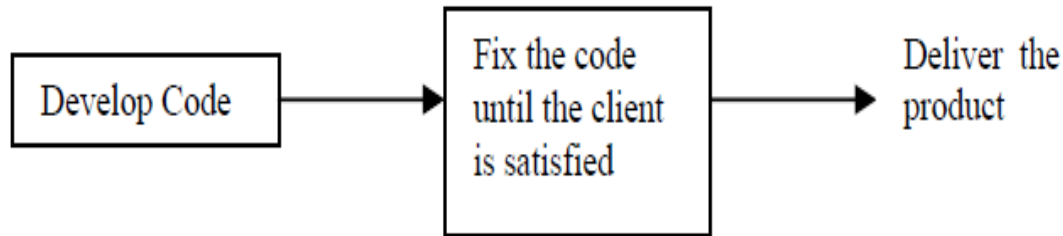
Software development models are used to develop or redesign software system. In a software development process, the main focus of an engineer is on **design, coding and testing**.



Build and Fix Model

Build and Fix Model

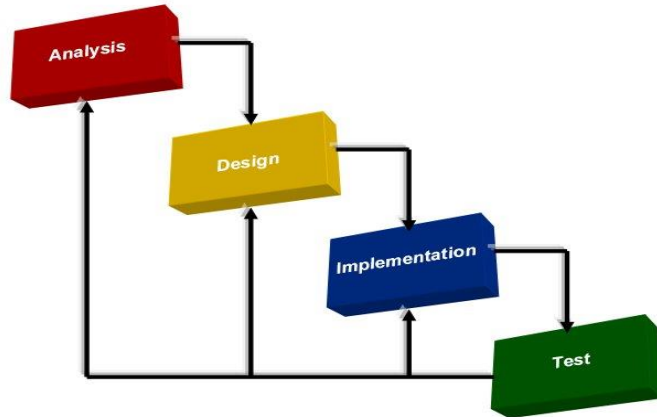
It is a simple two phase model. In one phase, code is developed and in another, code is fixed.



Waterfall Model

Waterfall Model

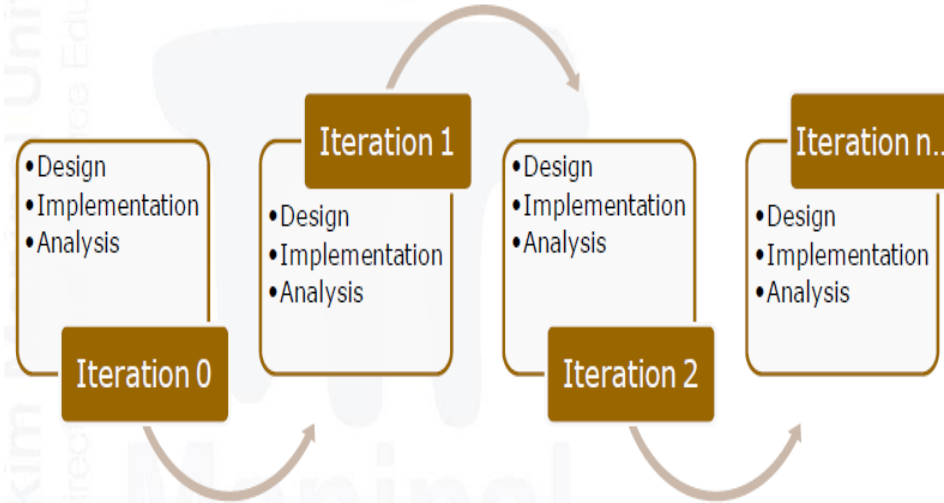
In this model, Once software is developed and is operational, then the feedback to various phases may be given.



Iterative Development Model

Iterative Development Model Iterative model develops the product in an increment model. At each increment, an extra feature is added until the complete product is implemented. This model creates the advantage of testing each increment rather than testing the entire product.

Iterative Development Model



Prototyping Model

Prototyping Model

In this model, a working model of actual software is developed initially. The prototype is just like a sample software having lesser functional capabilities and low reliability and it does not undergo through the rigorous testing phase.

The working prototype is given to the customer for operation. The customer, after its use, gives the feedback. Analyzing the feedback given by the customer, the developer refines, adds the requirements and prepares the final specification document.

Prototyping Model

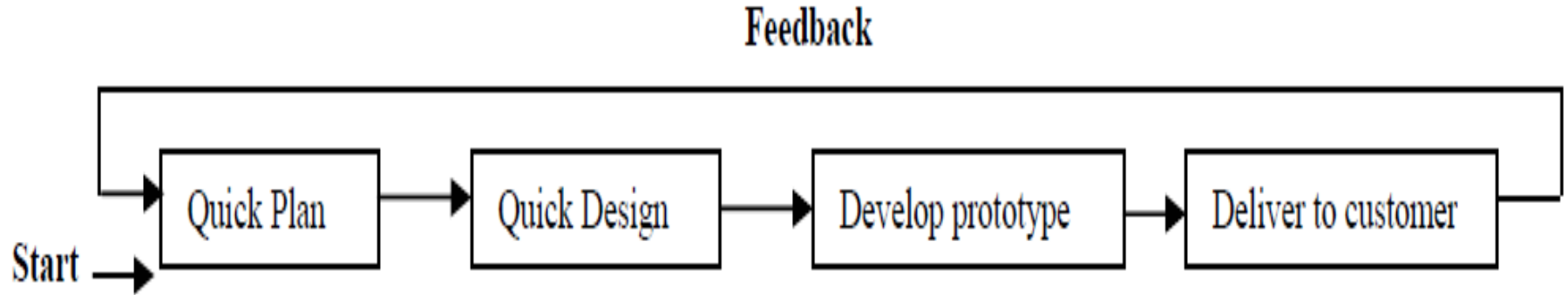


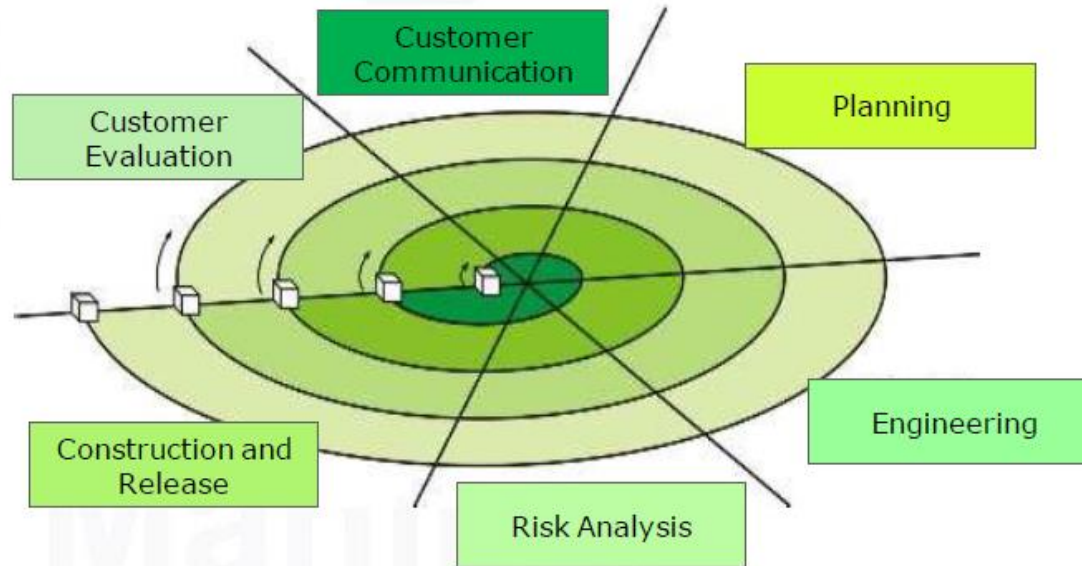
Figure 1.5: Prototyping model

Spiral Model

Spiral Model Spiral model couples the iterative nature of the prototype and the linear sequential model. The incremental release might be a paper model or prototype during early iterations. Increasingly more complete versions of product are released during later iterations.

Spiral Model

The six phases of this model are:



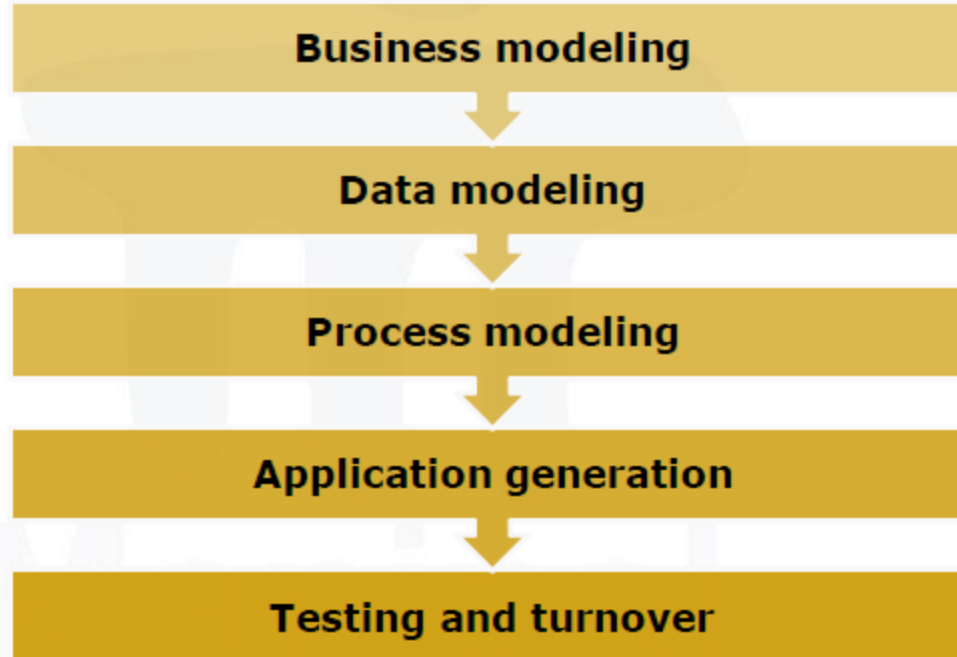
RAD Approach

Rapid Application Development (RAD Model)

RAD model is an upgrade of the linear sequential model in which the rapid development is accomplished by using component-based construction.

RAD Approach

RAD contains the following phases:



Capability Maturity Models

It defines five maturity levels as described below

Level 1 - The Initial Level

At the Initial Level, the organization typically does not provide a stable environment for developing and maintaining software. Such organizations frequently have difficulty making commitments that the staff can meet with an orderly engineering process, resulting in a series of crises. During a crisis, projects typically abandon planned procedures and revert to coding and testing. Success depends entirely on having an exceptional manager and a seasoned and effective software team.

Capability Maturity Models

Level 2 - The Repeatable Level

The organization satisfies all the requirements of level-1. At this level, basic project management policies and related procedures are established. The institutions achieving this maturity level learn with experience of earlier projects and reutilize the successful practices in on-going projects.

Level 3 (Defined): The organization satisfies all the requirements of level-2. At this maturity level, the software development processes are well defined, managed and documented. Training is imparted to staff to gain the required knowledge.

Capability Maturity Models

Level 4 (Managed): The organization satisfies all the requirements of level-3. At this level quantitative standards are set for software products and processes. The project analysis is done at integrated organizational level

Level 5 (Optimizing): The organization satisfies all the requirements of level-4. This is last level. The organization at this maturity level is considered almost perfect. At this level, the entire organization continuously works for process improvement with the help of quantitative feedback obtained from lower level. and collective database is created.

Software Process Technology

“A process defines who is doing what, when and how to reach a certain goal?” Software engineering is a field which combines process, methods and tools for the development of software.

The various steps (called phases) which are adopted in the development of this process are collectively termed as Software Development Life Cycle (SDLC).

Software Process Technology

In general, different phases of SDLC are defined as following:

- Requirements Analysis
- Design
- Coding
- Software Testing
- Maintenance.



Software Engineering

BLOCK 1 : Overview of Software Engineering

Unit-2

MCS
-213

[Principles of Software Requirements
Analysis]

Topics to be covered

- Engineering the Product
 - Requirements Engineering
 - Types of Requirements
 - Software Requirements Specification (SRS)
 - Structure of SRS
 - Requirements Gathering Tools
- Modeling the System Architecture
 - Elementary Modeling Techniques
 - Data Flow Diagrams
 - E-R Diagram
- Software Prototyping and Specification
- Software Metrics

Engineering The Product

Requirements Engineering

Requirements Engineering is the systematic use of proven principles, techniques and language tools for the cost effective analysis, documentation, on-going evaluation of user's needs and the specification of external behavior of a system to satisfy those user needs.

Engineering The Product

On the basis of their functionality, the requirements are classified into the following two types:

- i) **Functional requirements:** They define the factors like, I/O formats, storage structure, computational capabilities, timing and synchronization.
- ii) **Non-functional requirements:** They define the properties or qualities of a product including usability, efficiency, performance, space, reliability, portability etc.

Software Requirements Specification

- Software Requirements Specification (SRS) is a perfect detailed description of the behavior of the system to be developed. That is SRS document is an agreement between the developer and the customer covering the functional and non functional requirements of the software to be developed.
- SRS is considered as a contract between the customer and the developer.

Software Requirements Specification

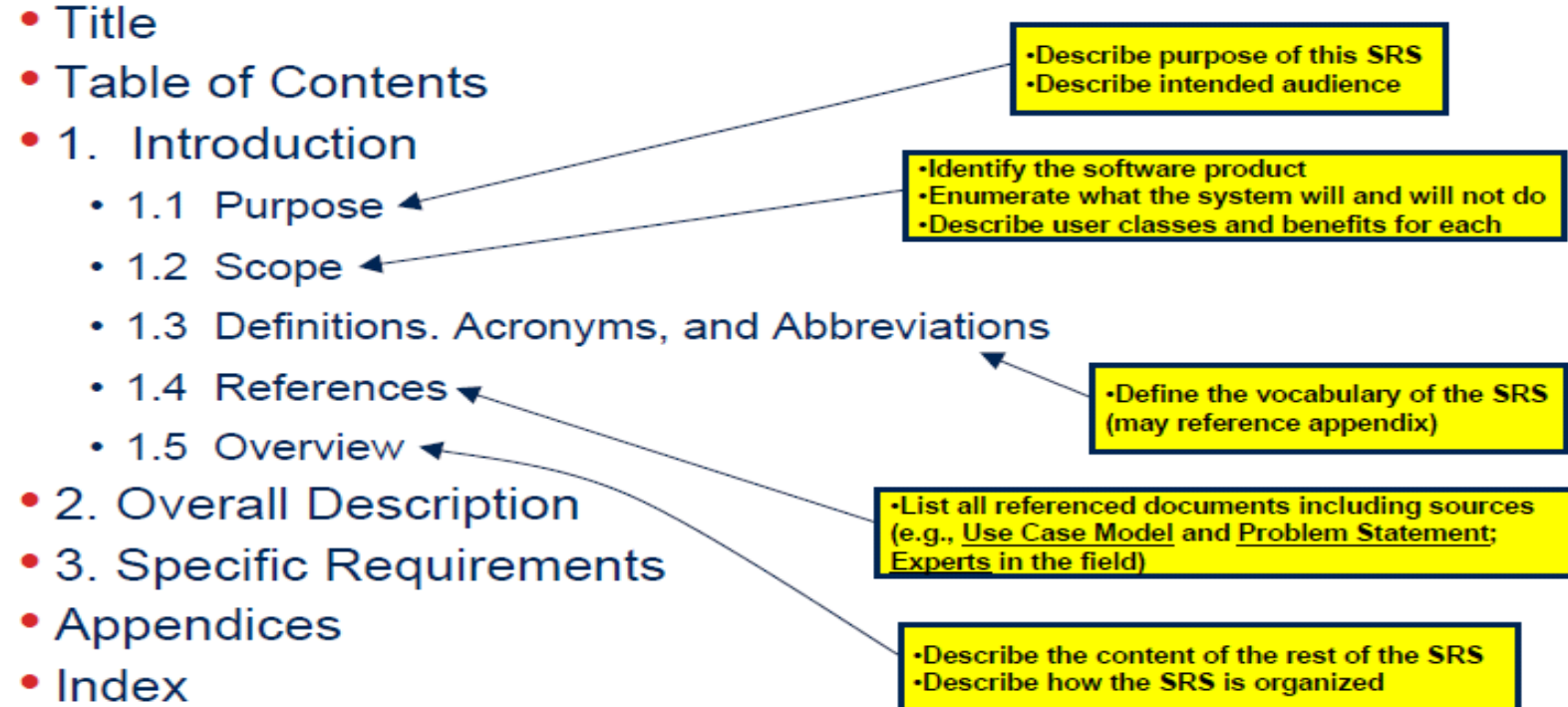
- This SRS document will be used for verifying whether all the functional and non functional requirements specified in the SRS are implemented in the product.
- The complete description of the functions to be performed by the software specified in the SRS will assist the potential users to determine if the software specified meets their needs or how the software must be modified to meet their needs.

Structure of SRS

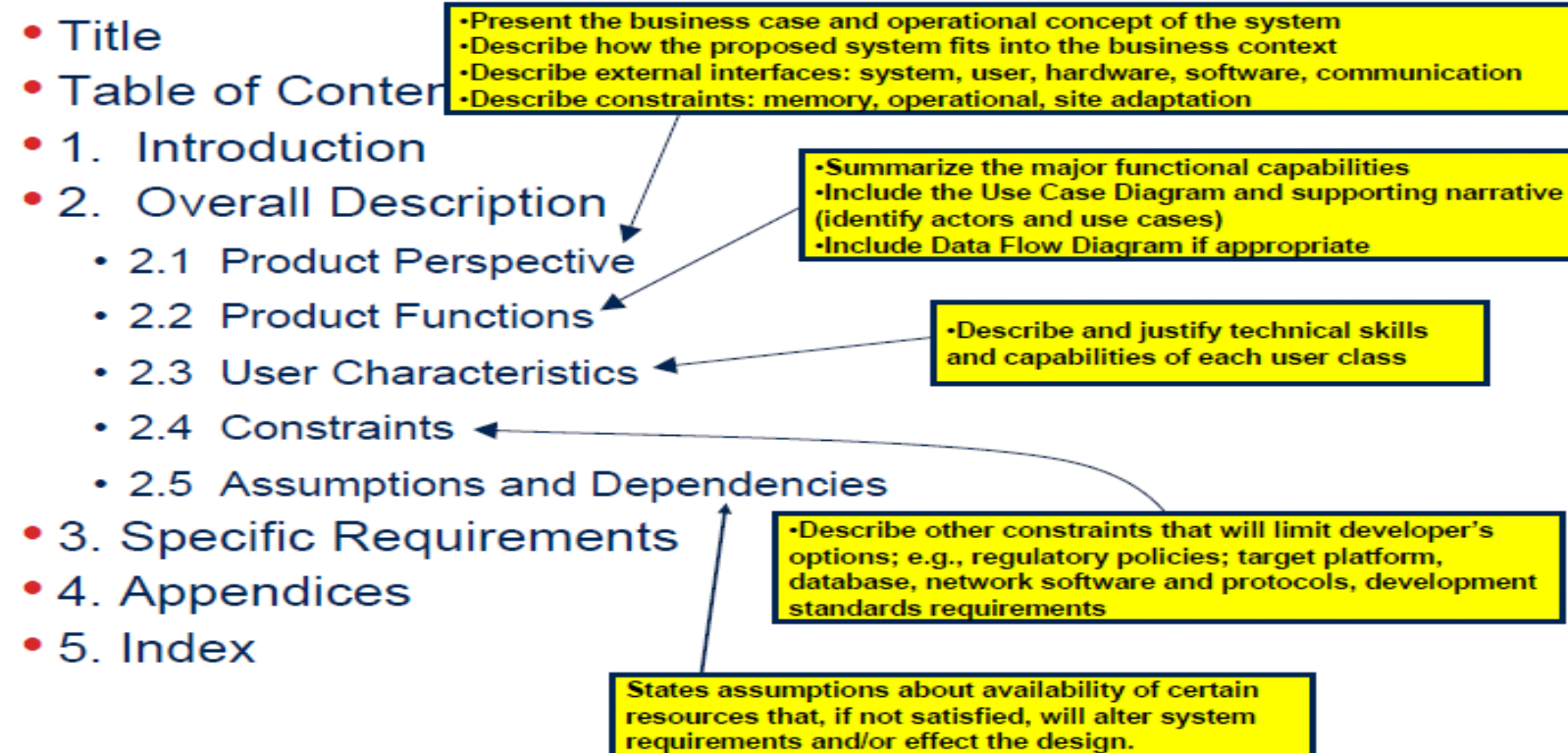
The SRS outline is given below:

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, acronyms, and abbreviations
 - 1.4 References
 - 1.5 Overview
2. Overall description
 - 2.1 Product perspective
 - 2.2 Product functions
 - 2.3 User characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and dependencies
3. Specific requirements
 - 3.1 External Interfaces
 - 3.2 Functional requirements
 - 3.3 Performance requirements
 - 3.4 Logical Database requirements
 - 3.5 Design Constraints
 - 3.6 Software system attributes
 - 3.7 Organising the specific requirements
 - 3.8 Additional Comments
4. Supporting information
 - 4.1 Table of contents and index
 - 4.2 Appendixes

Structure of SRS



Structure of SRS



Structure of SRS

• ...

- 1. Introduction
- 2. Overall Description
- 3. Specific Requirements
 - 3.1 External Interfaces
 - 3.2 Functions
 - 3.3 Performance Requirements
 - 3.4 Logical Database Requirements
 - 3.5 Design Constraints
 - 3.6 Software System Quality Attributes
 - 3.7 Object Oriented Models
- 4. Appendices
- 5. Index

•Detail all inputs and outputs
(complement, not duplicate, information presented in section 2)
•Examples: GUI screens, file formats

•Include detailed specifications of each
use case, including collaboration and
other diagrams useful for this purpose

•Include:
a) Types of information used
b) Data entities and their relationships

•Should include:
a) Standards compliance
b) Accounting & Auditing procedures

•The main body of requirements organized in a variety of
possible ways:
a) Architecture Specification
b) Class Diagram
c) State and Collaboration Diagrams
d) Activity Diagram (concurrent/distributed)

Requirements Gathering Tools

The requirements gathering is an art. The person who gathers requirements should have knowledge of what and when to gather information and by what resources.

The following four tools are primarily used for information gathering:

1. **Record review:** A review of recorded documents of the organization is performed. Procedures, manuals, forms and books are reviewed to see format and functions of present system. The search time in this technique is more.

Requirements Gathering Tools Continue...

2. **On site observation:** In case of real life systems, the actual site visit is performed to get a close look of system. It helps the analyst to detect the problems of existing system.

3. **Interview:** A personal interaction with staff is performed to identify their requirements. It requires experience of arranging the interview, setting the stage, avoiding arguments and evaluating the outcome.

Requirements Gathering Tools Continue...

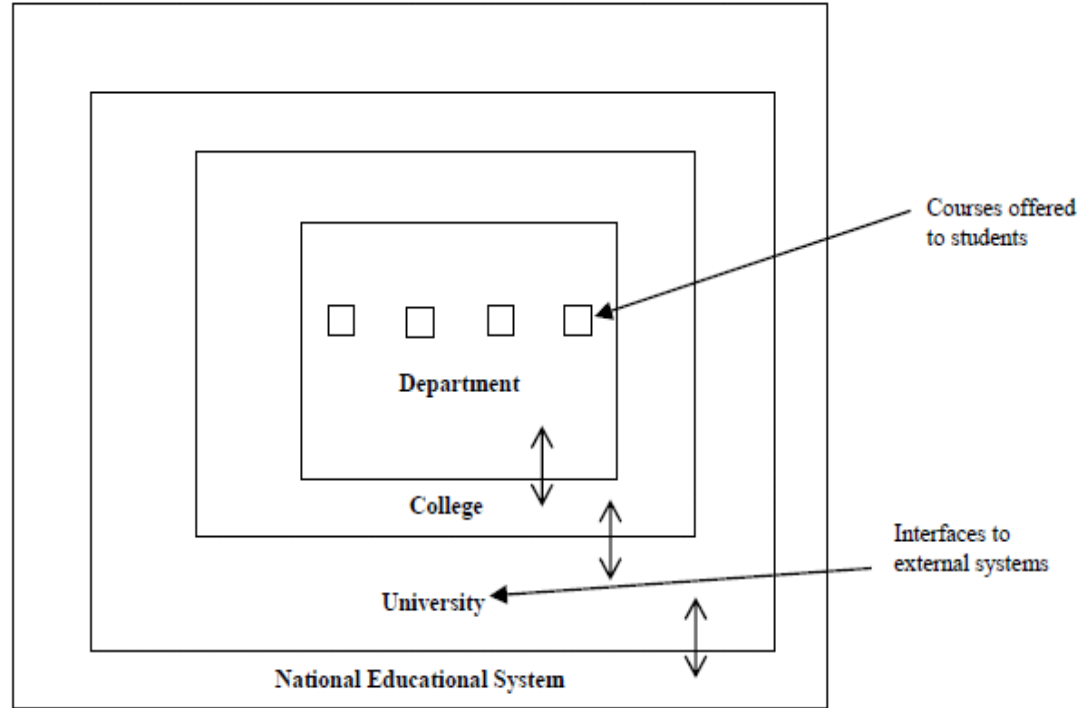
4. **Questionnaire:** It is an effective tool which requires less effort and produces a written document about requirements. It examines a large number of respondents simultaneously and gets customized answers. It gives person sufficient time to answer the queries and give correct answers.

System Modeling

System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.

1.Environmental model: It indicates environment in which system exists. Any big or small system is a sub-system of a larger system. For example, if software is developed for a college, then college will be part of University. If it is developed for University, the University will be part of national educational system

System Modeling



System Modeling

Behavioral models are used to describe the overall behavior of a system. The tools used to make this model are: Data Flow Diagrams (DFD), E-R diagrams, Data Dictionary & Process Specification.

Data Flow Diagrams (DFD)

DFDs model the system from a functional perspective. 1 Tracking and documenting how the data associated with a process is helpful to develop an overall understanding of the system. 1 Data flow diagrams may also be used in showing the data exchange between a system and other systems in its environment

Data Flow Diagram

DFD's can represent the system at any level of abstraction. DFD of "0" level views entire software element as a single bubble with indication of only input and output data. Thus, "0" level DFD is also called as Context diagram. Its symbols are shown in *Figure 2.4*.

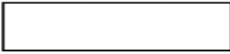
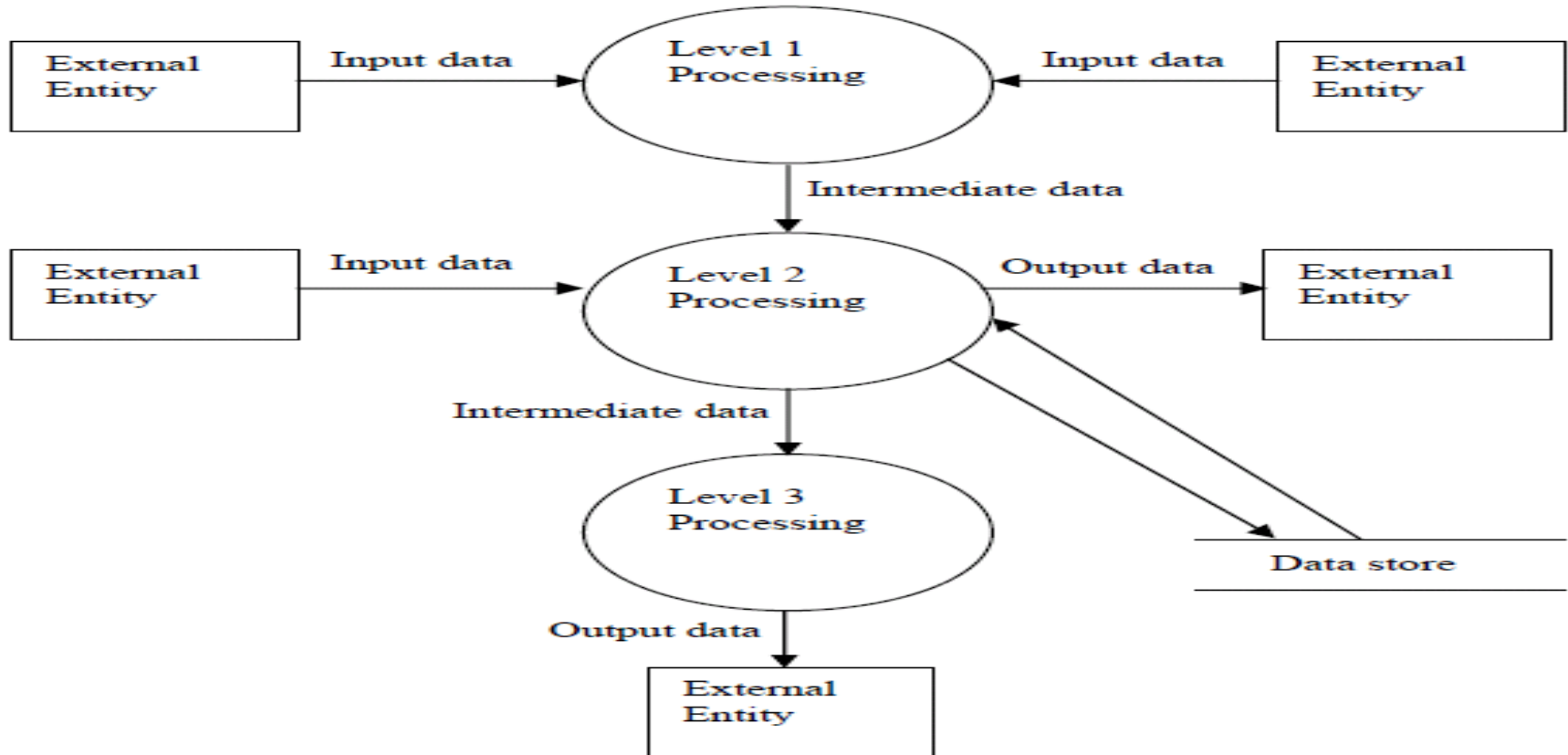
Symbol	Name	Description
	Data Flow	Represents the connectivity between various processes
	Process	Performs some processing of input data
	External Entity	Defines source or destination of system data. The entity which receives or supplies information.
	Data Store	Repository of data

Figure 2.4: Symbols of a data flow diagram

Data Flow Diagram Example



E-R Diagram

Entity-relationship (E-R) diagram is detailed logical representation of data for an organization. It is data oriented model of a system whereas DFD is a process oriented model. It has three main components – data entities, their relationships and their associated attributes.

Entity: It is most elementary thing of an organization about which data is to be maintained.

Relationship: Entities are connected to each other by relationships. It indicates how two entities are associated. A diamond notation with name of relationship represents as written inside.

E-R Diagram

Cardinality & Optionally: The cardinality represents the relationship between two entities. Consider the one to many relationship between two entities – class and student.

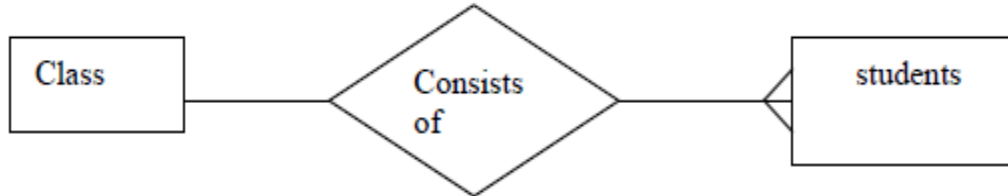
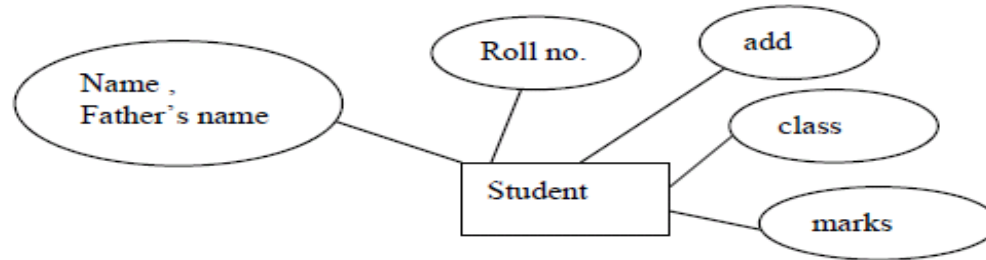


Figure 2.6: One to Many cardinality relationship

E-R Diagram

Attributes: Attribute is a property or characteristic of an entity that is of interest to the organization. It is represented by oval shaped box with name of attribute written inside it.



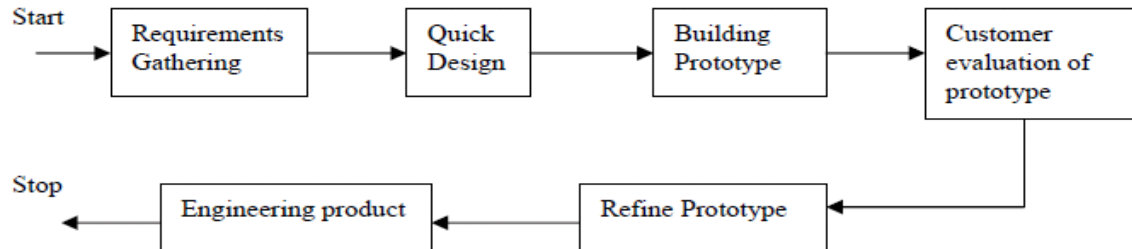
Software Prototyping And Specification

A prototype may be categorized as follows:

1. A paper prototype, which is a model depicting human machine interaction in a form that makes the user understand how such interaction, will occur.
2. A working prototype implementing a subset of complete features.
3. An existing program that performs all of the desired functions but additional features are added for improvement.

Software Prototyping And Specification

Prototype is developed so that customers, users and developers can learn more about the problem. Thus, prototype serves as a mechanism for identifying software requirements.



Software Metrics

Software metrics are objective and quantitative data that is used to measure different characteristics of a software system or the software development process. Using the software metrics, software engineer measures software processes, and the requirements for that process. The software measures are done according to the following parameters:

Software Metrics

- The objective of software and problems associated with current activities,
- The cost of software required for relevant planning relative to future projects,
- Testability and maintainability of various processes and products,
- Quality of software attributes like reliability, portability and maintainability,



Software Engineering

BLOCK 1 : Overview of Software Engineering

MCS-213
Block-1

Unit-3 [Software Design]

Topics to be Covered

Software Design

Data Design

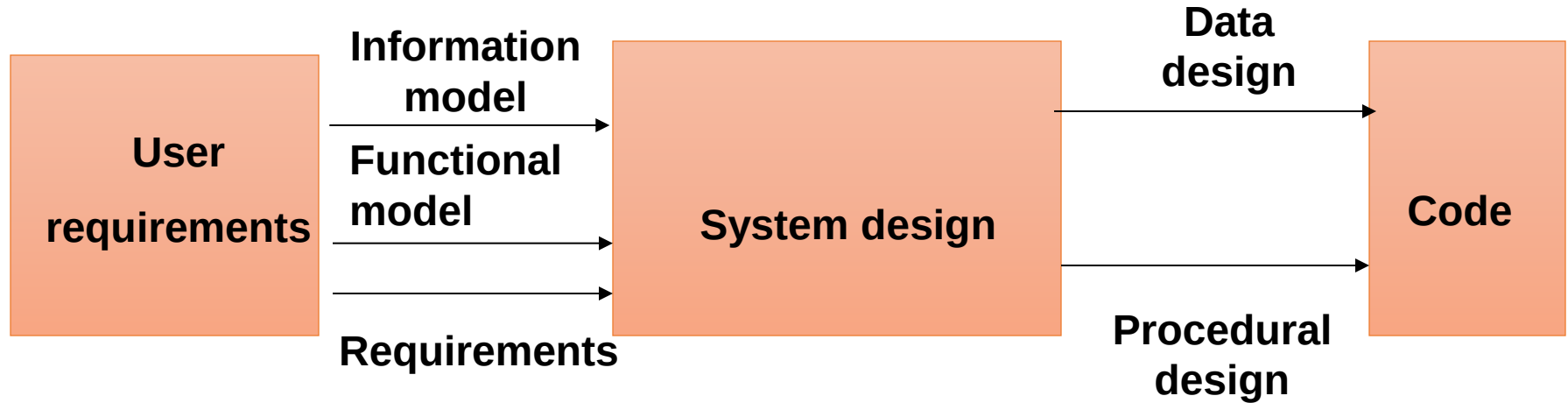
Architectural Design

Modular Design

Interface Design

Design of Human Computer Interface

Software Design



Data Design

- Data design is the first and the foremost activity of system Design.
- Data describes a real-world information resource that is important for the application.
- Data describes various entities like customer, people, asset, student record etc.
- The primary objective of the data design is to select logical representation of data items identified in requirement analysis phase.

Architectural Design

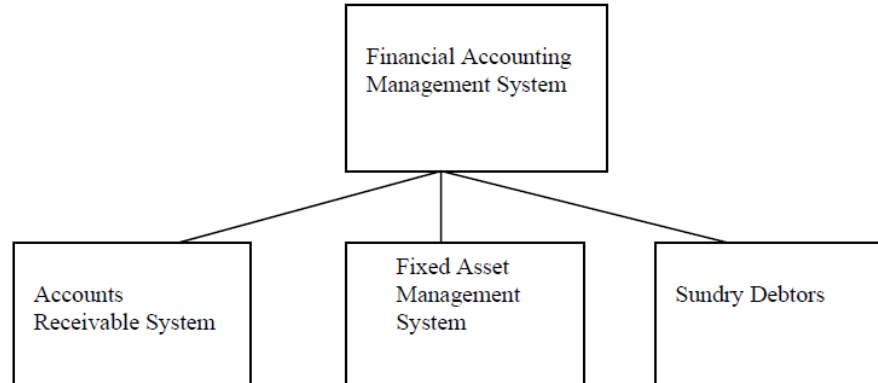
The objective of architectural design is to develop a model of software architecture which gives an overall organization of program module in the software product.

Architectural design defines organization of program components. It does not provide the details of each components and its implementation.

Architectural Design

The Objective of architectural design is also to control relationship between modules.

One module may control another module or may be controlled by another module.

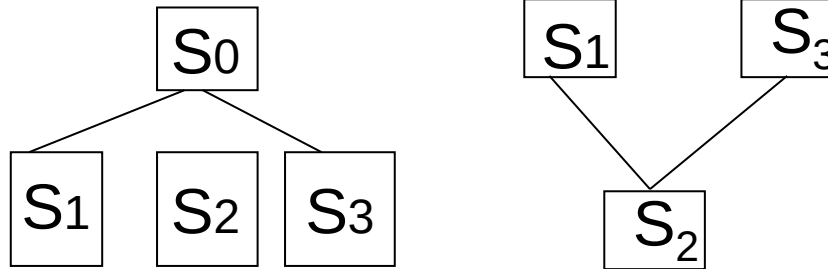


Architectural Design

The organization of module can be represented through a tree like structure. The number of components which controls a said component is called fan-in i.e. the number of incoming edges to a component. The number of components that are controlled by the module is called fan-out i.e. the number of outgoing edges.

Architectural Design

S_0 controls three components, hence the fan-out is 3. S_2 is controlled by two components, namely, S_1 and S_3 , hence the fan-in is 2.



Modular Design

The concept of modular approach has been derived from the fundamental concept of “divide and conquer”.

Modularity is the only attribute of a software product that makes it manageable and maintainable.

Dividing the software to modules helps software developer to work on independent modules that can be later integrated to build the final product.

It encourages parallel development efforts thus saving time and cost.

Modular Design

There should be a balance between number of modules and integration cost. Although, more numbers of modules make the software product more manageable and maintainable at the same time, as the number of modules increases, the cost of integrating the modules also increases.

Modular Design

While modularity is good for software quality, independence between various modules are even better for software quality and manageability, independence is measured by two parameters

- ❖ Cohesion
- ❖ Coupling

Cohesion

Cohesion is a measure of functional strength of a software module. This is the degree of interaction between statement in a module.

Highly cohesive system requires little interaction with external module.

There are several types of cohesion arranged from bad to best.

Cohesion

Coincidental : This is worst form of cohesion, where a module performs a number of unrelated task.

Logical : modules perform series of action, but are selected by a calling module

Procedural : Modules perform a series of steps. The elements in the module must take-up single control sequence and must be executed in a specific order.

Communicational : All elements in the module is executed on the same data set and produce same output data set.

Coupling

Coupling is defined as degree to which a module interacts and communicates with another module to perform certain task.

If one module relies on another the coupling is said to be high.

Low level of coupling means a module does not have to concerned with the internal details of another module.

Coupling

Types of Coupling

(From best (lowest level of coupling) to worst (high level of coupling)

Data Coupling: Modules interact through parameters. Module X passes parameter A to Module Y.

Stamp coupling: Modules shares composite data structure.

Control coupling: One module control logic flow of another module. For exp : passing a flag to another module which determines the sequence of action to be performed in the other module depending on the value of flag such as true or false.

Coupling

External coupling: Modules shares external data in a common format. Mostly used in communication protocols and device interfaces.

Content coupling: One module modifies the data of another module.

Sequential: Output from one element is input to some other element in module. Such modules operates on some data structure.

Functional: Modules contain elements that perform exactly one function.

Disadvantages of Coupling

Difficult to reuse. Dependent modules must be included.

Difficult to understand the function of a module in isolation.

Interface Design

Interface design is the most important part of software design.
The following are the elements of good interface design.

Goal and the intention of task must be identified

Use of consistent color scheme, message and terminologies helps

Develop standards for good interface design and stick to it

Use icons where ever possible to provide appropriate message

Allow user to undo the current command

Provide sensitive help to guide the user

Provide Key-board short cut

Design Principles

Recoverability:

The system should provide some resilience to user errors and allow the user to recover from errors. This might include an undo facility, confirmation of destructive actions, 'soft' deletes, etc.

User guidance:

Some user guidance such as help systems, on-line manuals, etc. should be supplied

User diversity

Interaction facilities for different types of user should be supported. For example, some users have seeing difficulties and so larger text should be available

Thank you!

Software Engineering

BLOCK 1

Overview of Software Engineering

MCS-213

Block-1

Unit-4[Software Quality Assurance]

Topics to be covered

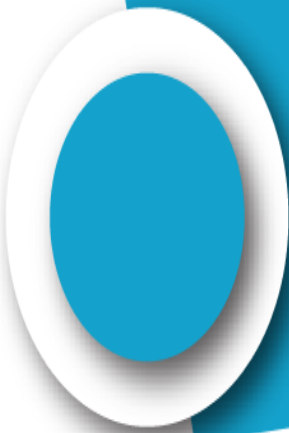
- 
- ☐ Introduction
 - ☐ Software Quality
 - ☐ Formal Technical Review
 - ☐ Software Reliability
 - ☐ Software Quality Standards

Introduction

Software quality assurance is a series of activities undertaken throughout the software engineering process. It comprises the entire gamut of software engineering life cycle. The objective of the software quality assurance process is to produce high quality software that meets customer requirements through a process of applying various procedures and standards.

Quality software is reasonably bug-free, delivered on time and within budget, meets requirements and is maintainable.

Software Quality



Good quality software satisfies both explicit and implicit requirements. Software quality is a complex mix of characteristics and varies from application to application and the customer who requests for it.

Attributes of Quality

Auditability : The ability of software being tested against conformance to standard.

Compatibility : The ability of two or more systems or components to perform their required functions while sharing the same hardware or software environment.

Completeness: The degree to which all of the software's required functions and design constraints are present and fully developed in the requirements specification, design document and code.

Attributes of Quality

Consistency: The degree of uniformity, standardization, and freedom from contradiction among the documents or parts of a system or component.

Correctness: The degree to which a system or component is free from faults in its specification, design, and implementation. The degree to which software, documentation meet specified requirements.

Attributes of Quality

Modularity : The degree to which a system or computer program is composed of discrete components such that a change to one component has minimal impact on other components.

Predictability: The degree to which the functionality and performance of the software are determinable for a specified set of inputs.

Robustness: The degree to which a system or component can function correctly in the presence of invalid inputs or stressful environmental conditions.

Attributes of Quality

Structuredness : The degree to which the SDD (System Design Document) and code possess a definite pattern in their interdependent parts.

Testability: The degree to which a system or component facilitates the establishment of test criteria and the performance of tests to determine whether those criteria have been met.

Attributes of Quality

Traceability: The degree to which a relationship can be established between two or more products of the development process. The degree to which each element in a software development product establishes its reason for existing (e.g., the degree to which each element in a bubble chart references the requirement that it satisfies). For example, the system's functionality must be traceable to user requirements.

Attributes of Quality

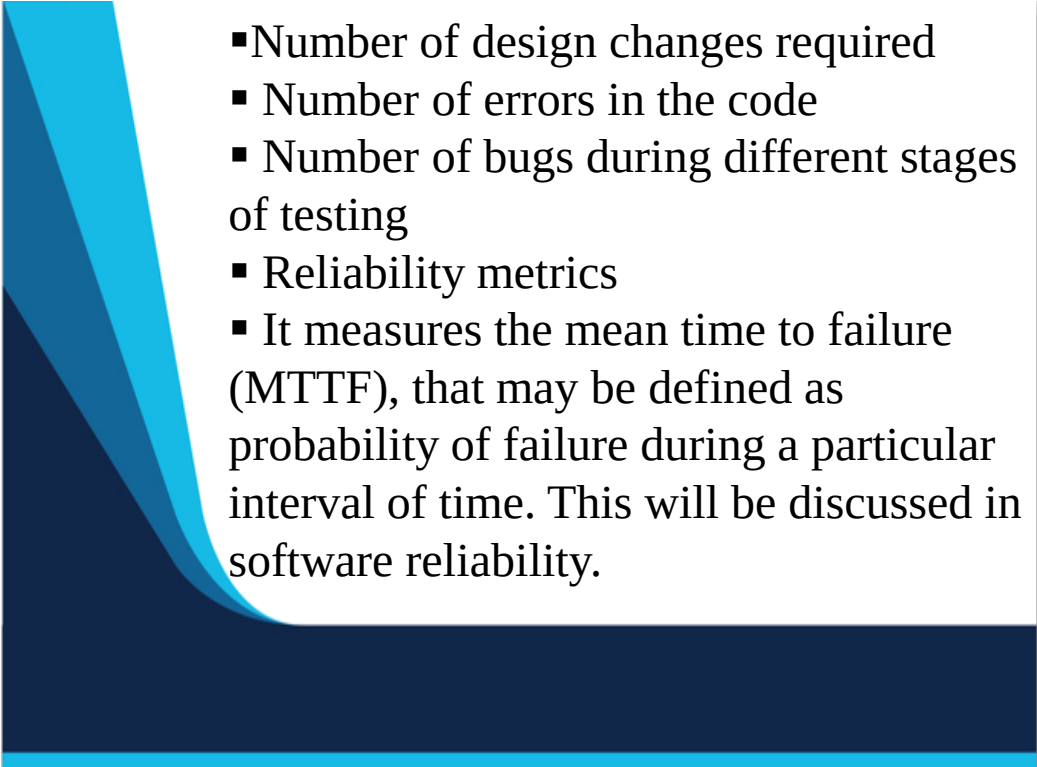
Understandability: The degree to which the meaning of the SRS, SDD, and code are clear and understandable to the reader.

Verifiability : The degree to which the SRS, SDD, and code have been written to facilitate verification and testing.

Causes of error in Software

- Misinterpretation of customers' requirements/communication
- Incomplete/erroneous system specification
- Error in logic
- Not following programming/software standards
- Incomplete testing
- Inaccurate documentation/no documentation
- Deviation from specification
- Error in data modeling and representation.

Characteristics of software quality

- 
- Number of design changes required
 - Number of errors in the code
 - Number of bugs during different stages of testing
 - Reliability metrics
 - It measures the mean time to failure (MTTF), that may be defined as probability of failure during a particular interval of time. This will be discussed in software reliability.

Maintainability metrics

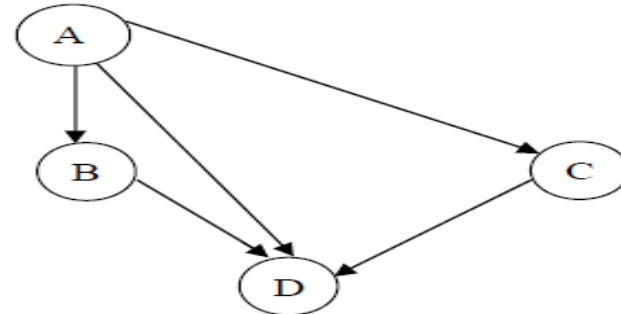
Complexity metrics are used to determine the maintainability of software. The complexity of software can be measured from its control flow.

$$V(G) = e - n + 2 \text{ where}$$

$V(G)$: is called Cyclomatic complexity of the program

e = number of edges

n = number of nodes



Process Of Software Quality

- Defines the requirements for software controlled system fault/failure detection, isolation, and recovery;
- Reviews the software development processes and products for software error prevention and/ or controlled change to reduced functionality states;
- Defines the process for measuring and analyzing defects as well as reliability and maintainability factors.

Formal Technical Review

Software review can't be defined as a filter for the software engineering process.

- Reviews are basically of two types, informal technical review and formal technical review.

- Informal Technical Review : Informal meeting and informal desk checking.

- Formal Technical Review: A formal software quality assurance activity through various approaches such as structured walkthroughs, inspection, etc.

Formal Technical Review

Formal technical review is a software quality assurance activity performed by software engineering practitioners to improve software product quality. The product is scrutinized for completeness, correctness, consistency, technical feasibility, efficiency, and adherence to established standards and guidelines by the client organization.

Formal Technical Review

➤ *Verification* : Verification generally involves reviews to evaluate whether correct methodologies have been followed by checking documents, plans, code, requirements, and specifications. This can be done with checklists, walkthroughs, etc.

➤ *Validation* : Validation typically involves actual testing and takes place after verifications are completed.

Objectives of Formal Technical Review

- To uncover errors in logic or implementation
- To ensure that the software has been represented accruing to predefined standards
- To ensure that software under review meets the requirements
- To make the project more manageable.
- The schedule of the meeting and its agenda reach the members well in advance
- Members review the material and its distribution
- The reviewer must review the material in advance.

Software Concept and Initiation phase

- Involvement of SQA team in both writing and reviewing the project management plan in order to assure that the processes, procedures, and standards identified in the plan are appropriate, clear, specific, and auditable.
- Have design constraints been taken in to account?
- Whether best alternatives have been selected.

Software Requirements Analysis Phase

During the software requirements phase, review assures that software requirements are complete, testable, and properly expressed as functional, performance, and interface requirements. The output of a software requirement analysis phase is Software Requirements Specification (SRS). SRS forms the major input for review.

Compatibility

- Does the interface enables compatibility with external interfaces?
- Whether the specified models, algorithms, and numerical techniques are compatible?

Software Requirements Analysis Phase

Completeness

- Does it include all requirements relating to functionality, performance, system constraints, etc.?
- Does SRS include all user requirements?

Consistency

- Are the requirements consistent with each other? Is the SRS free of contradictions?
- Whether SRS uses standard terminology and definitions throughout.

Software Requirements Analysis Phase

Correctness

- Are the requirements adequate and correctly indicated?
- Is there justification for the design/implementation constraints?

Traceability

- Are the requirements traceable to top level?
- Is there traceability from the next higher level of specification?

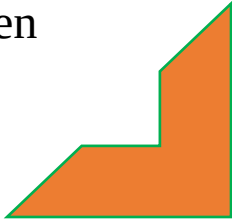
Verifiability and Testability

- Are validation criteria complete?

Software Design Phase

- 
- Reviewers should be able to determine whether or not all design features are consistent with the requirements.

Completeness

- Whether all the requirements have been addressed in the SDD?
 - Have the software requirements been properly reflected in software architecture?
- 

Software Design Phase

Consistency

- Are input and output formats consistent to the extent possible?
- Are the designs for similar or related functions are consistent?

Correctness

- Does the SDD conform to design documentation standards?
- Does the design perform only that which is specified in the SRS?

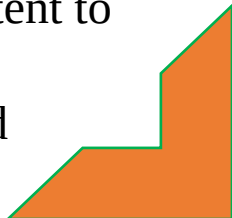
Software Design Phase



Completeness

- Whether all the requirements have been addressed in the SDD?
- Have the software requirements been properly reflected in software architecture?

Consistency

- Are input and output formats consistent to the extent possible?
 - Are the designs for similar or related functions are consistent?
- 

Software Design Phase

Correctness

- Does the SDD conform to design documentation standards?
- Does the design perform only that which is specified in the SRS?

Modifiability

- The modules are organized such that changes in the requirements only require changes to a small number of modules.

Software Design Phase

Traceability

- Is the SDD traceable to requirements in SRS?
- Does the SDD show mapping and complete coverage of all requirements and design constraints in the SRS?

Verifiability

- Does the SDD describe each function using well-defined notation so that the SDD can be verified against the SRS

Review of Source Code

- Completeness
- Consistency
- Correctness
- Modifiability
- Traceability
- Understandability
- Software testing & implementation phase
- Installation

Software Reliability

Software reliability is typically measured per a unit of time, whereas probability of failure is generally time independent. These two measures can be easily related if you know the frequency with which inputs are executed per unit of time. Mean-time-to-failure (MTTF) is the average interval of time between failures; this is also sometimes referred to as Mean-time-before-failure.

Software Reliability

Definitions of Software reliability

- IEEE Definition: The probability that software will not cause the failure of a system for a specified time under specified conditions. The probability is a function of the inputs to and use of the system in the software. The inputs to the system determine whether existing faults, if any, are encountered.
- The ability of a program to perform its required functions accurately and reproducibly under stated conditions for a period of time.

Software Reliability Models

A number of models have been developed to determine software defects/failures. All these models describe the occurrence of defects/failures as a function of time. This allows us to define reliability. These models are based on certain assumptions which can be described as below:

Software Reliability Models

- The failures are independent of each other, i.e., one failure has no impact on other failure(s).
- The inputs are random samples.
- Failure intervals are independent and all software failures are observed.
- Time between failures is exponentially distributed.

Software quality standards

Standards	Title
ISO/IEC 90003	Software Engineering. Guidelines for the Application of ISO 9001:2000 to Computer Software
ISO 9001:2000	Quality Management Systems - Requirements
ISO/IEC 14598-1	Information Technology-Evaluation of Software Products-General Guide
ISO/IEC 9126-1	Software Engineering - Product Quality - Part 1: Quality Model
ISO/IEC 12119	Information Technology-Software Packages-Quality Requirements and Testing
ISO/IEC 12207	Information Technology-Software Life Cycle Processes
ISO/IEC 14102	Guideline For the Evaluation and Selection of CASE Tools
IEC 60880	Software for Computers in the Safety Systems of Nuclear Power Stations
IEEE 730	Software Quality Assurance Plans
IEEE 730.1	Guide for Software Assurance Planning
IEEE 982.1	Standard Dictionary of Measures to produce reliable software
IEEE 1061	Standard for a Software Quality Metrics Methodology
IEEE 1540	Software Life Cycle Processes - Risk Management
IEEE 1028	Software review and audits
IEEE 1012	Software verification and validation plans

Thank
You

Software Engineering

BLOCK 2: Software Project Management

MCS-213

Block-2

Unit-5 [Software Project Planning]

Topics to be cover

- Different Types of Project Metrics
- Software Project Estimation
 - Estimating the Size
 - Estimating Effort
 - Estimating Schedule
 - Estimating Cost
- COCOMO Model
- Putnam's Model
- Statistical Model
- Automated Tools for Estimation

Metrics

A **software metric** is a quantitative measure of a degree to which a software system or process possesses some property.

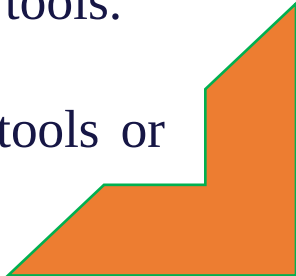
Metrics deal with measurement of the software process and the software product. Metrics quantify the characteristics of a process or a product. Metrics are often used to estimate project cost and project schedule.

Metrics

Goal Of Metrics

To improve product quality and development-team productivity.
Concerned with productivity and quality measures
Measures of SW development output as function of effort and time
•Measures of usability

Need For Project Metrics

- 
1. To indicate the quality of the product.
 2. To assess the productivity of the people who produce the product.
 3. To assess the benefits derived from new software engineering methods and tools.
 4. To form a baseline for estimation.
 5. To help justify requests for new tools or additional training.
- 

Types Of Project Metrics

Metrics can be broadly divided into two categories:

- Product Metrics
- Process Metrics

Product Metrics

Product metrics are measures of the software product at any stage of its development, from requirements to installed system. Product metrics may measure:

- The complexity of the software design
- The size of the final program
- The number of pages of documentation produced

Lines of Code

Lines of Code(LOC) : LOC metric is possibly the most extensively used for measurement of size of a program. The reason is that LOC can be precisely defined. LOC may include executable source code and non-executable program code like comments etc.

Lines of Code

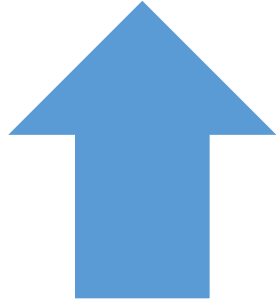
Module	Effort in man-months	LOC
Module 1	3	24,000
Module 2	4	25,000

Looking at the data above we have a direct measure of the size of the module in terms of LOC. We can derive a productivity metrics from the above primitive metrics i.e., LOC.

Productivity of a person = $\text{LOC} / \text{man-month}$

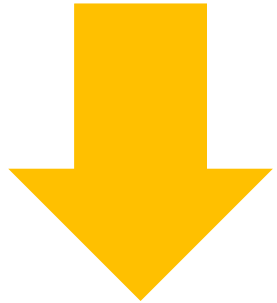
Quality = $\text{No. of defects} / \text{LOC}$

Lines of Code



Advantages of Size Oriented Metrics

LOC can be easily counted



Disadvantages of Size Oriented Metrics

LOC measures are language dependent, programmer dependent
Their use in estimation requires a lot of detail which can be difficult to achieve.

Function Metrics

Function point : Function point metrics instead of LOC measures the functionality of the program. Based on “functionality” delivered by the software. Functionality is measured indirectly using a measure called *function point*. Function points (FP) - derived using an empirical relationship based on countable measures of software & assessments of software complexity.

Function Metrics

In a Function point analysis, the following features are considered:

External inputs : A process by which data crosses the boundary of the system. Data may be used to update one or more logical files.

External outputs : A process by which data crosses the boundary of the system to outside of the system. It can be a user report or a system log report.

Function Metrics

External user inquires : A count of the process in which both input and output results in data retrieval from the system. These are basically system inquiry processes.

Internal logical files : A group of logically related data files that resides entirely within the boundary of the application software and is maintained through external input as described above.

Function Metrics

<u>Parameter</u>	<u>Count</u>	<u>Simple</u>	<u>Average</u>	<u>Complex</u>	
Inputs	x	3	4	6	=
Outputs	x	4	5	7	=
Inquiries	x	3	4	6	=
Files	x	7	10	15	=
Interfaces	x	5	7	10	=

Count-total (raw FP)

Benefits Function Metrics

J2EE is that standard that also provides portability of code because it is based on Java technology and standard based Java programming APIs **Benefits of Using Function Points**

- Function points can be used to estimate the size of a software application correctly irrespective of technology, language and development methodology.

Benefits Function Metrics

- User understands the basis on which the size of software is calculated as these are derived directly from user required functionalities.
- Function points can be used to track and monitor projects.
- Function points can be calculated at various stages of software development process and can be compared.

Software Project Estimation



Software project estimation is the process of estimating various resources required for the completion of a project. Effective software project estimation is an important activity in any software development project.

Software Project Estimation

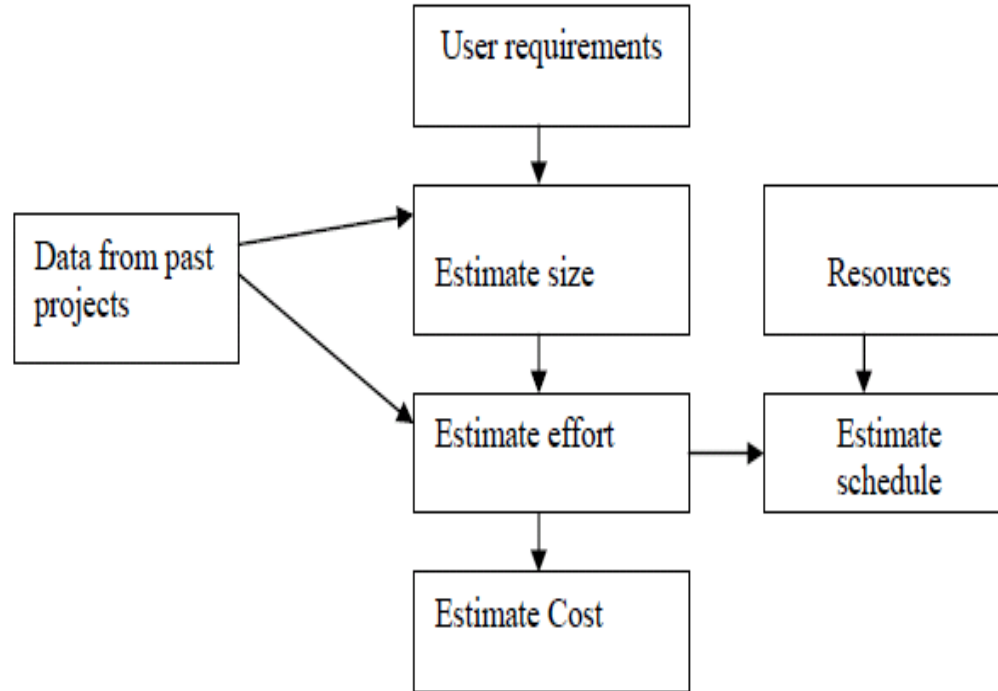
Underestimating software project and understaffing it often leads to low quality deliverables, and the project misses the target deadline leading to customer dissatisfaction and loss of credibility to the company. On the other hand, overstaffing a project without proper control will increase the cost of the project and reduce the competitiveness of the company.

Software Project Estimation

The four basic steps in software project estimation are:

- 1) Estimating the size of project. There are many procedures available for estimating the size of a project which are based on quantitative approaches like estimating Lines of Code or estimating the functionality requirements of the project called Function point.
- 2) Estimate the effort in person-months or person-hours.
- 3) Estimate the schedule in calendar months.
- 4) Estimate the project cost in dollars (or local currency)

Software Project Estimation Process



Software Project Estimation



Estimating the size

Estimating the size of the software to be developed is the very first step to make an effective estimation of the project. Customer's requirements and system specification forms a baseline for estimating the size of a software.



Software Project Estimation

- The ways to estimate project size can be through past data from an earlier developed system. This is called estimation by analogy.
- The other way of estimation is through product feature/functionality. The system is divided into several subsystems depending on functionality, and size of each subsystem is calculated.

Software Project Estimation

Estimating effort

Once the size of software is estimated, the next step is to estimate the effort based on the size. The estimation of effort can be made from the organizational specifics of software development life cycle.

Software Project Estimation

- 
- The best way to estimate effort is based on the organization's own historical data of development process.
 - If the project is of a different nature which requires the organization to adopt a different strategy for development then different models based on algorithmic approach can be devised to estimate effort.
- 

Software Project Estimation

Estimating Schedule

The next step in estimation process is estimating the project schedule from the effort estimated. The schedule for a project will generally depend on human resources involved in a process. Efforts in man-months are translated to calendar months.

- Schedule estimation in calendar-month can be calculated using the following model [McConnell]:
- Schedule in calendar months = $3.0 * (\text{man-months})^{1/3}$
- The parameter 3.0 is variable, used depending on the situation which works best for the organization.

Software Project Estimation

Estimating Cost

Cost estimation is the next step for projects. The cost of a project is derived not only from the estimates of effort and size but from other parameters such as hardware, travel expenses, telecommunication costs, training cost etc. should also be taken into account.

COCOMO Model

COCOMO Model

COCOMO stands for Constructive Cost Model. It was introduced by Barry Boehm. It provides the following three level of models:

- **Basic COCOMO** :We use single variable COCOMO, where a quick and rough estimate of cost of a software project is to be done. However, it is not very accurate.

COCOMO Model

- **Intermediate COCOMO:** The static multi-variable model takes care of the cost drivers which could not be handled in the single variable model.
- **Detailed COCOMO :** This model computes development effort and cost which incorporates all characteristics of intermediate level with assessment of cost implication on each step of development (analysis, design, testing etc.)

COCOMO Model

This model may be applied to three classes of software projects as given below:

- Organic : Small size project. A simple software project where the development team has good experience of the application
- Semi-detached : An intermediate size project and project is based on rigid and semi-rigid requirements.

COCOMO Model

- Embedded : The project developed under hardware, software and operational constraints.
- In the COCOMO model, the development effort equation assumes the following form: $E = aS^b m$
 - here **a** and **b** are constraints that are determined for each model.
 - E = Effort , S = Value of source in LOC , m = multiplier that is determined from a set of 15 cost driver's attributes.

Putnam's model

L. H. Putnam developed a dynamic multivariate model of the software development process based on the assumption that distribution of effort over the life of software development is described by the Rayleigh-Norden curve.

$$P = Kt \exp(t^2/2T^2) / T^2$$

P = No. of persons on the project at time 't'

K = The area under Rayleigh curve which is equal to total life cycle effort

T = Development time

Statistical Model

C.E. Walston and C.P. Felix developed a simple empirical model of software development effort with respect to number of lines of code. In this model, LOC is assumed to be directly related to development effort as given below:

$E = a L^b$ Where L = Number of Lines of Code (LOC)

E = total effort required

a and b are parameters obtained from regression analysis of data. The final equation is of the following form:

$$E = 5.2 L^{0.91}$$

Design Patterns MVC

This pattern divides the appliance into three parts that are dependent and connected to every other.

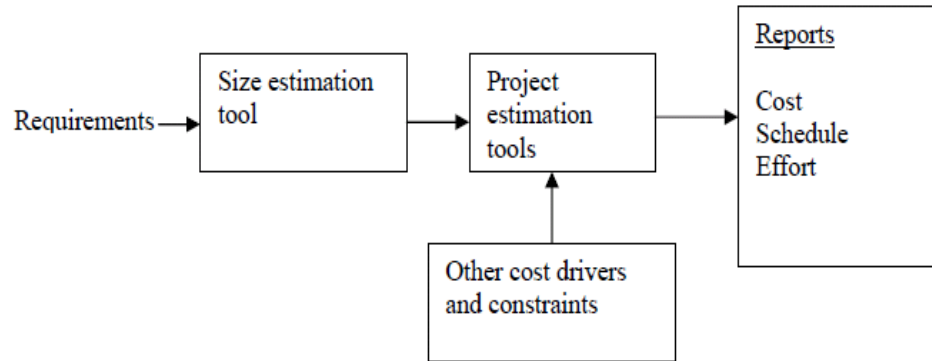
These designs are wont to distinguish the presentation of knowledge from how the info is accepted from the user to the info shown.

These design patterns became common within the use of web applications and for developing GUIs.

Automated Tools For Estimation

There are more than dozens of estimation tools available. But, all of them have the following common characteristics:

Quantitative estimates of project size (e.g., LOC).
Estimates such as project schedule, cost.



Thank
You

Software Engineering

BLOCK 2: Software Project Management

MCS-213

Block-2

Unit-6 [Risk mgmt. and Project Scheduling]

Topics to be covered

- ❖ Identification of Software Risks
- ❖ Monitoring of Risks
- ❖ Management of Risks
 - Risk Management
 - Risk Avoidance
 - Risk Detection
 - Risk Control
 - Risk Recovery
- ❖ Task Set for the Project
- ❖ Choosing the Tasks of Software Engineering
- ❖ Scheduling Methods

Identification Of Software Risks

Risk

Any anticipated unfavorable event or circumstances that occur while the project is underway.

If the risk become true

It can hamper the successful and timely completion of a project.

Therefore, it is necessary to anticipate and identify different risks.

Different Types of Software risks

1 Lack of Knowledge

2 Poor knowledge of tools

3 Management Issues

4 Updates in the hardware resources

5 Customer Risks

6 External Risks

7 Commercial Risks

Management Of Risks

Risk management plays an important role in ensuring that the software product is error free. Firstly, risk management takes care that the risk is avoided, and if it not avoidable, then the risk is detected, controlled and finally recovered.

The risk manager does scheduling of risks.

Management Of Risks

1. Risk Avoidance

- a. Risk anticipation
- b. Risk tools

2. Risk Detection

- a. Risk analysis
- b. Risk category
- c. Risk prioritization

Management Of Risks

3. Risk Status

- a. Risk pending
- b. Risk resolution
- c. Risk not solvable

4. Risk Recovery

- a. Full
- b. Partial
- c. Extra/alternate features

Management Of Risks

1. Risk Avoidance

Risk can be avoided by choosing an alternative approach with lower risk

- Risk Anticipation: Various risk anticipation rules are listed according to standards from previous projects experience.
- Risk tools: Risk tools are used to test whether the software is risk free. The tools have built-in data base of available risk areas and can be updated depending upon the type of project.

Management Of Risks

2. Risk Detection

The risk detection algorithm detects a risk and it can be categorically stated as :

Risk Analysis: In this phase, the risk is analyzed with various hardware and software parameters as probabilistic occurrence (pr), weight factor (wf) (hardware resources, lines of code, persons), risk exposure ($pr * wf$). Risk category: risk identification can be from various factors like persons involved in the team, management issues, customer specification etc. Once proper category is identified, priority is given depending upon the urgency of the product.

Management Of Risks

Risk prioritization: depending upon the entries of the risk analysis table, the maximum risk exposure is given high priority and has to be solved first

3. Risk control

Once the prioritization is done, the next step is to control various risks as follows:

- Risk pending: According to the priority, low priority risks are pushed at the end of the queue with a view of various resources (hardware, man power, software) and in case it takes more time their priority is made higher.

Management Of Risks

- Risk Resolution: Risk manager makes a strong resolve how to solve the risk.
- Risk not solvable: If a risk takes more time and more resources, it is notified to the customer, and the team member proposes an alternate solution. There is a slight variation in the customer specification after consultation.

Management Of Risks

4. Risk recovery

- Full : The risk analysis table is scanned and if the risk is fully solved, then corresponding entry is deleted from the table.
- Partial : The risk analysis table is scanned and due to partially solved risks, the entries in the table are updated and thereby priorities are also updated.
- Extra/alternate features : Sometimes it is difficult to remove some risks, and in that case, we can add a few extra features, which solves the problem.

Task Set For The Project

- 1 **Technical staff expertise**
- 2 **Customer satisfaction**
- 3 **Technology update**
- 4 **Full or partial implementation of the project**
- 5 **Time allocation**
- 6 **Module binding**
- 7 **Validation and Verification**

Choosing The Tasks Of Software Engineering

Once the task set has been defined, the next step is to choose the tasks for software project.

Scope : Overall scope of the project.

Scheduling and planning : Scheduling of various modules and their milestones

Technology used : Latest hardware and software used.

Choosing The Tasks Of Software Engineering

Customer interaction : Obtaining feedback from the customer.

Constraints and limitations : Various constraints in the project and how they can be solved. Limitations in the modules and how they can be implemented .

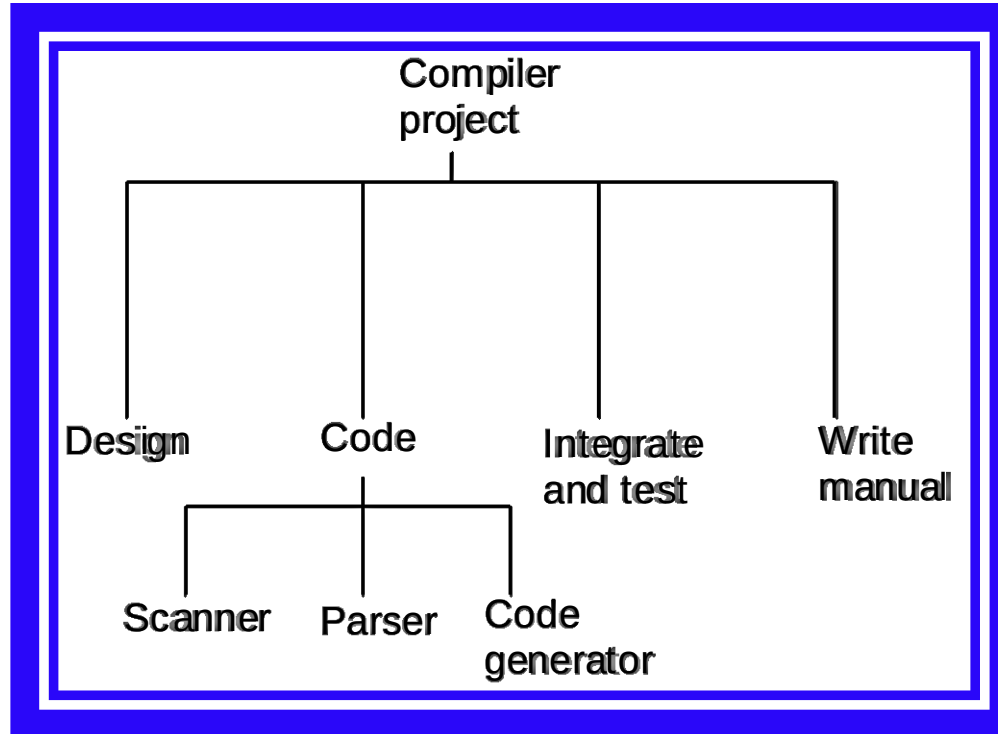
Risk Assessment : Risk involved in the project with respect to limitations in the technology and resources.

Scheduling Methods

Scheduling can be done with resource constraint or time constraint in mind. Depending upon the project, scheduling methods can be static or dynamic in implementation.

Work Breakdown Structure : WBS describes a break down of project goal into intermediate goals each in turn broken down in a hierarchical structure. The work breakdown structure shows the overall breakup flow of the project and does not indicate any parallel flow.

Work Breakdown Structure

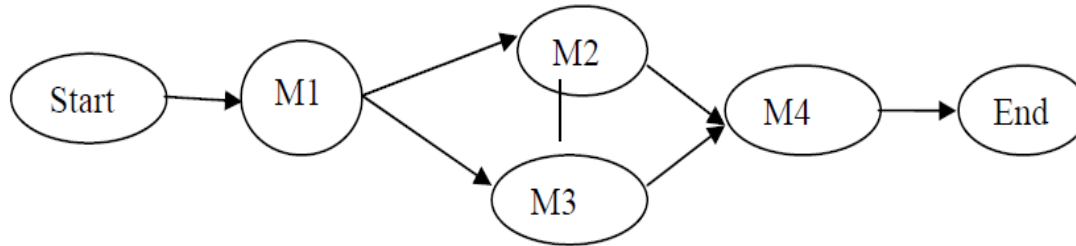


Flow Graph

Various modules are represented as nodes with edges connecting nodes. Dependency between nodes is shown by flow of data between nodes.

Cycles are not allowed in the graph.

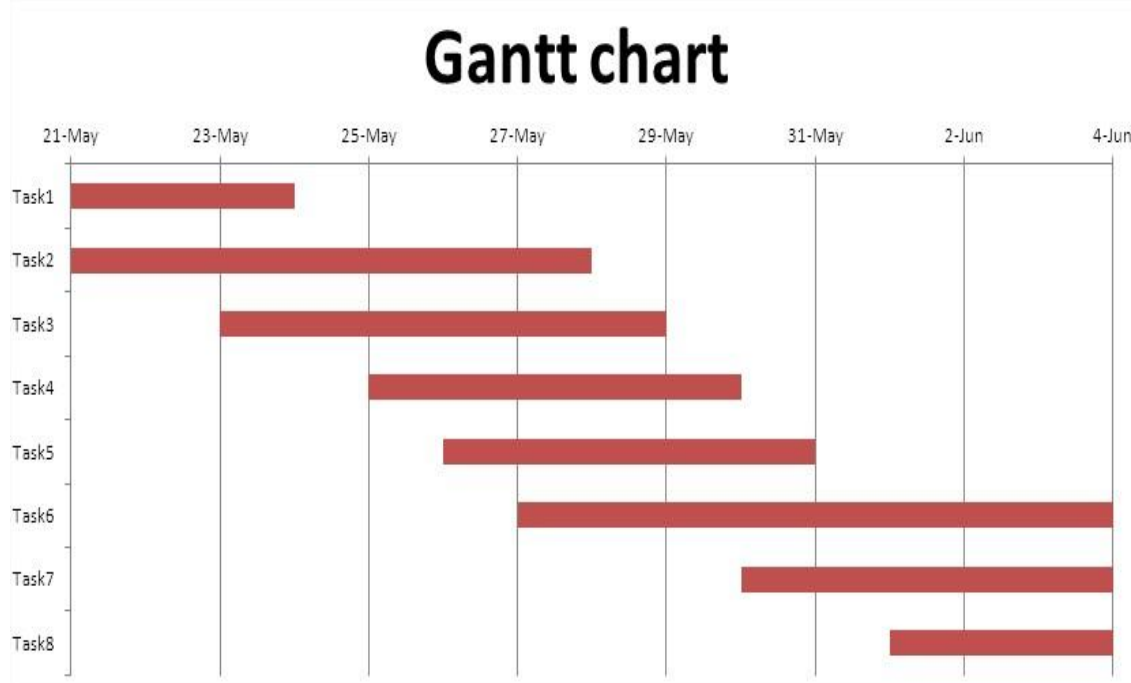
Start and end nodes indicate the source and terminating nodes



Gantt Chart or Time Line Charts

- It is a project control technique
- Defined by Henry L. Gantt
- Used for several purposes, including scheduling, budgeting & resource planning.
- A tabular form is maintained where rows indicate the tasks with milestones and the horizontal bars that spans across columns indicate duration of the task.

Gantt Chart or Time Line Charts

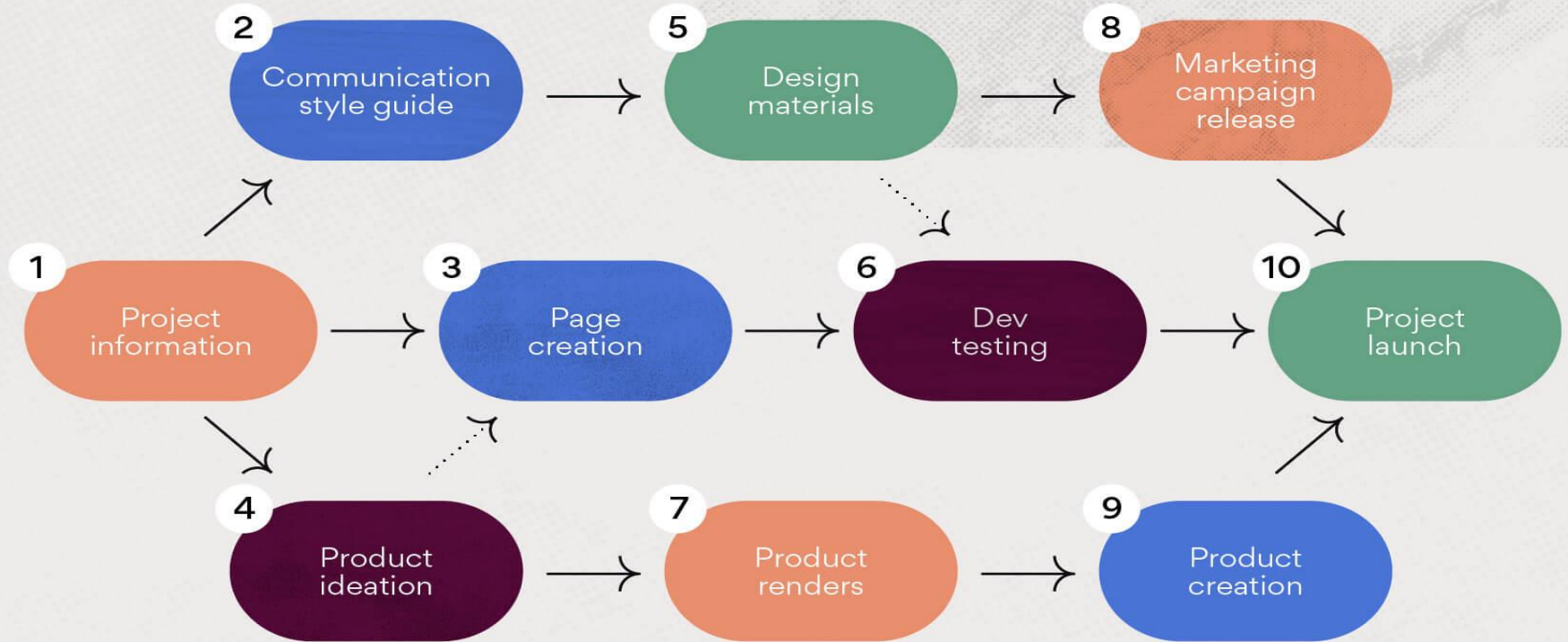


Program Evaluation Review Technique

Mainly used for high-risk projects with various estimation parameters. For each module in a project, duration is estimated as follows:

1. Time taken to complete a project or module under normal conditions.
2. Time taken to complete a project or module with minimum time (all resources available).
3. Time taken to complete a project or module with maximum time (resource constraints).
4. Time taken to complete a project from previous related history.

PERT chart example



*Thank
you*



Software Engineering

BLOCK 2: Software Project Management

MCS-213

Block-2

Unit-7 [Software Testing]

Topics to be Covered

- Basic Terms used in Testing
 - Input Domain
 - Black Box and White Box testing Strategies
 - Cyclomatic Complexity
- Testing Activities
- Debugging
- Testing Tools

What is Testing?

“Testing is an activity in which a system or component is executed under specified conditions; the results are observed and recorded and an evaluation is made of some aspect of the system or component.

What is Testing?



Testing is a process of executing a program with the intent of finding an error.



A good test case is one that has a high probability of finding an as-yet-undiscovered error.

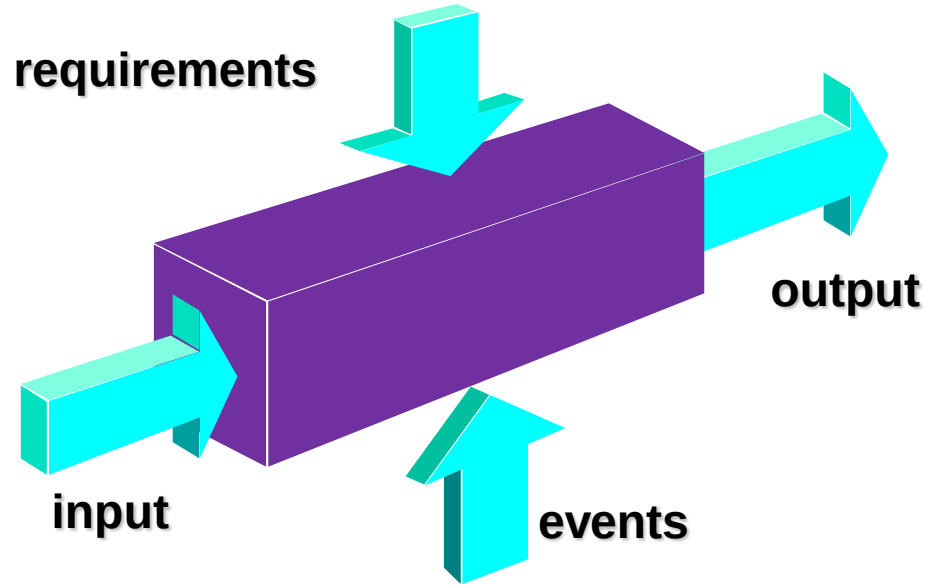


A successful test is one that uncovers an as-yet-undiscovered error.

Test Case Principle

- 1 The tests should be traceable to customer requirements
- 2 The tests should be planned in advance i.e., before the testing begins
- 3 The tests should follow the Pareto principle
- 4 The tests should begin from individual components and then progress toward testing the entire system
- 6 The tests should be conducted by an independent third party, to result in an effective testing.

Black Box Testing



Black Box Testing

Black-box testing is another software testing method that deals with the functional requirements of the software. Black-box testing is also known as behavioral testing and is a complementary approach that is likely to uncover a different class of errors than white-box methods.

Black Box Testing

- Test cases are derived from formal specification of the system.
- Test case selection can be done without any reference to the program design or code.
- Only tests the functionality and features of the program, Not the internal operation.

Black Box Testing

Graph-Based Testing Methods

The software testing begins by creating a graph of objects and their relationships. Once the graph has been created, each object and relationship is exercised. Then, a series of tests are devised to uncover errors.

Black Box Testing

Equivalence partitioning

This method divides the input domain of a program into classes of data from which test cases can be derived. It tries hard to define test case, which can uncover classes of errors, thus reducing the number of test cases needed.

Black Box Testing

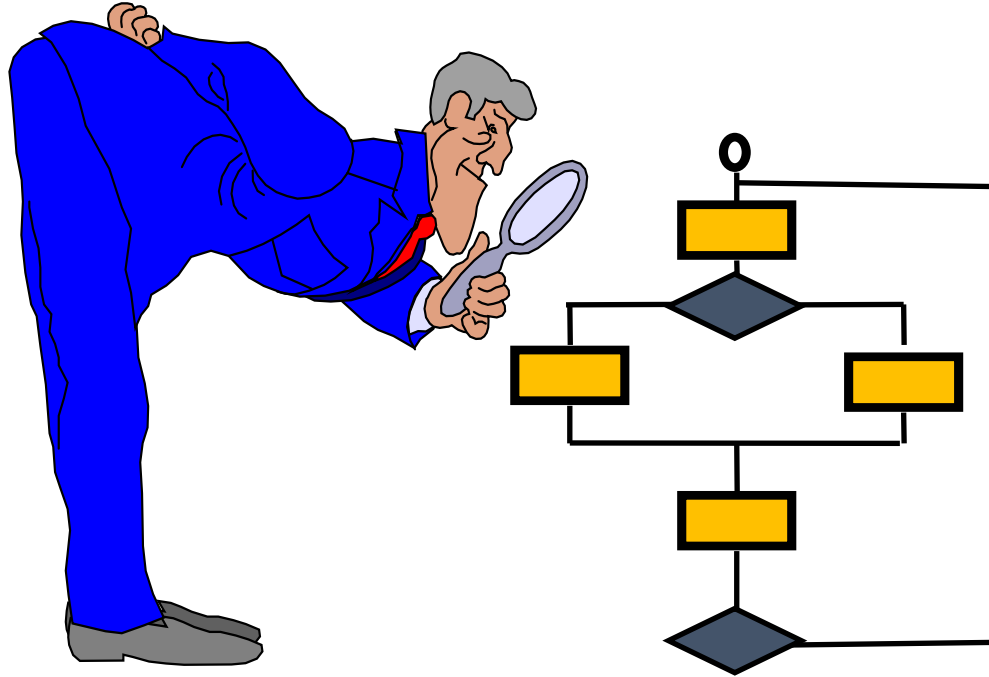
Boundary value analysis

Boundary Value Analysis (BVA) is a test case design method that complements equivalence partitioning. Instead of selecting any element of an equivalence class, it selects the test cases at the "edges" of the class. BVA derives test cases from the output domain, instead of input. Boundary value analysis leads to a selection of test cases that exercise bounding values. As, most of the errors occur at boundaries of the input domain, rather in the center. Following are the guidelines for Boundary value analysis (BVA), which are similar to that of equivalence partitioning:

Black Box Testing

- If input condition = $(a, \dots, b)$, Bounded by a and b then, test cases should be designed with values a and b and just above and just below a and b .
- If input condition = Number of values, then test cases should be developed that exercise the minimum and maximum numbers. Values just above and below minimum and maximum are also tested.
- If internal program data structures have prescribed boundaries, then test cases should be developed that exercise the data structure at its boundary.

White Box Testing

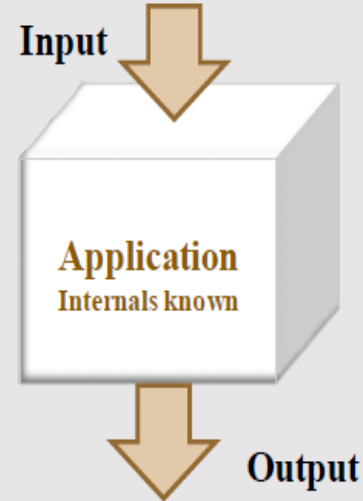


White Box Testing

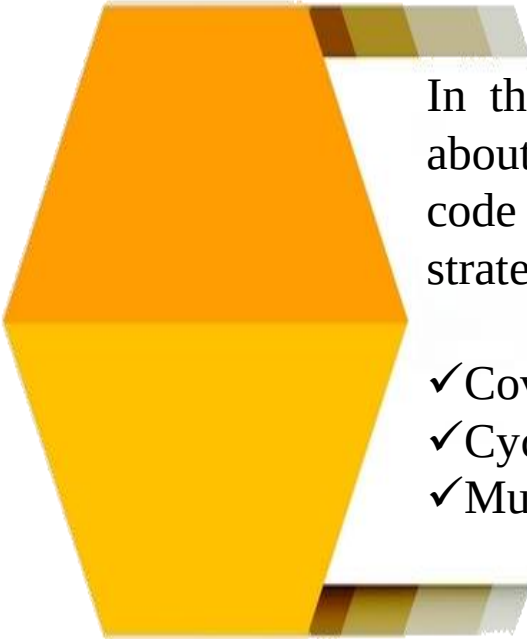
White-box testing is one of the software testing methods that checks the internal working of an application. It uses control structure of the procedural design to derive test cases. White-box testing is also known as glass-box testing.

White Box Testing

The programmer has an access to program codes. He can analyze the codes (look inside the box) and then proceed with the testing process, based on the information that he obtained.



Methods for White box testing strategies



In this approach, complete knowledge about the internal structure of the source code is required. For White-box testing strategies, the methods are:

- ✓ Coverage Based Testing
- ✓ Cyclomatic Complexity
- ✓ Mutation Testing

Coverage Based Testing



Coverage based testing works by choosing test cases according to well-defined 'coverage' criteria. The more common coverage criteria are the following.

➤ **Statement Coverage or Node Coverage:** Every statement of the program should be exercised at least once.

➤ **Branch Coverage or Decision Coverage:** Every possible alternative in a branch or decision of the program should be exercised at least once. For if statements, this means that the branch must be made to take on the values true or false.



Coverage Based Testing

- **Decision/Condition Coverage:** Each condition in a branch is made to evaluate to both true and false and each branch is made to evaluate to both true and false.
- **Multiple condition coverage:** All possible combinations of condition outcomes within each branch should be exercised at least once.
- **Path coverage:** Every execution path of the program should be exercised at least once.

Cyclomatic Complexity

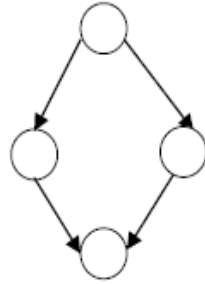
Control flow graph (CFG)

A control flow graph describes the sequence in which different instructions of a program get executed. It also describes how the flow of control passes through the program. In order to draw the control flow graph of a program, we need to first number all the statements of a program. The different numbered statements serve as nodes of the control flow graph. An edge from one node to another node exists if the execution of the statement representing the first node can result in the transfer of control to the other node. Following structured programming constructs are represented as CFG:

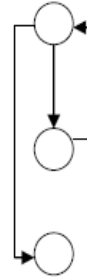
Control Flow Graph



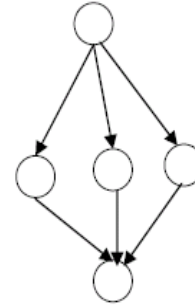
Sequence



If
else



While
Loop



case

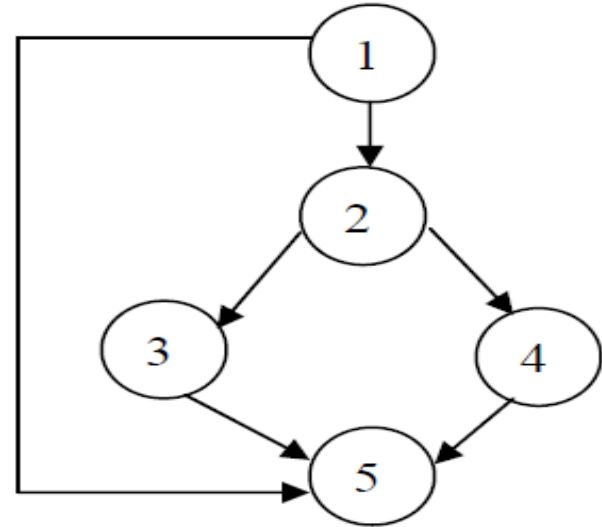
Cyclomatic Complexity

Cyclomatic Complexity: This technique is used to find the number of independent paths through a program. If CFG of a program is given, the Cyclomatic complexity $V(G)$ can be computed as: $V(G) = E - N + 2$, where N is the number of nodes of the CFG and E is the number of edges in the CFG. For the example 4, the Cyclomatic Complexity = $6 - 5 + 2 = 3$.

Cyclomatic Complexity

The following are the properties of Cyclomatic complexity:

- $V(G)$ is the maximum number of independent paths in graph G
- Inserting and deleting functional statements to G does not affect $V(G)$
- G has only one path if and only if $V(G) = 1$.



Mutation Testing

Mutation Testing is a powerful method for finding errors in software programs. In this technique, multiple copies of programs are made, and each copy is altered; this altered copy is called a mutant. Mutants are executed with test data to determine whether the test data are capable of detecting the change between the original program and the mutated program.

Mutation Testing

If we run a mutated program, there are two possibilities:

1. The results of the program were affected by the code change and the test suite detects it. We assumed that the test suite is perfect, which means that it must detect the change. If this happens, the mutant is called a killed mutant.
2. The results of the program are not changed and the test suite does not detect the mutation. The mutant is called an equivalent mutant.

Testing Activities

Requirements Analysis: Testing should begin in the requirements phase of the software development life cycle (SDLC).

Design Analysis: During the design phase, testers work with developers in determining what aspects of a design are testable and under what parameters should the testers work.

Test Planning: Test Strategy, Test Plan(s).

Test Development: Test Procedures, Test Scenarios, Test Cases, Test Scripts to use in testing software.

Testing Activities

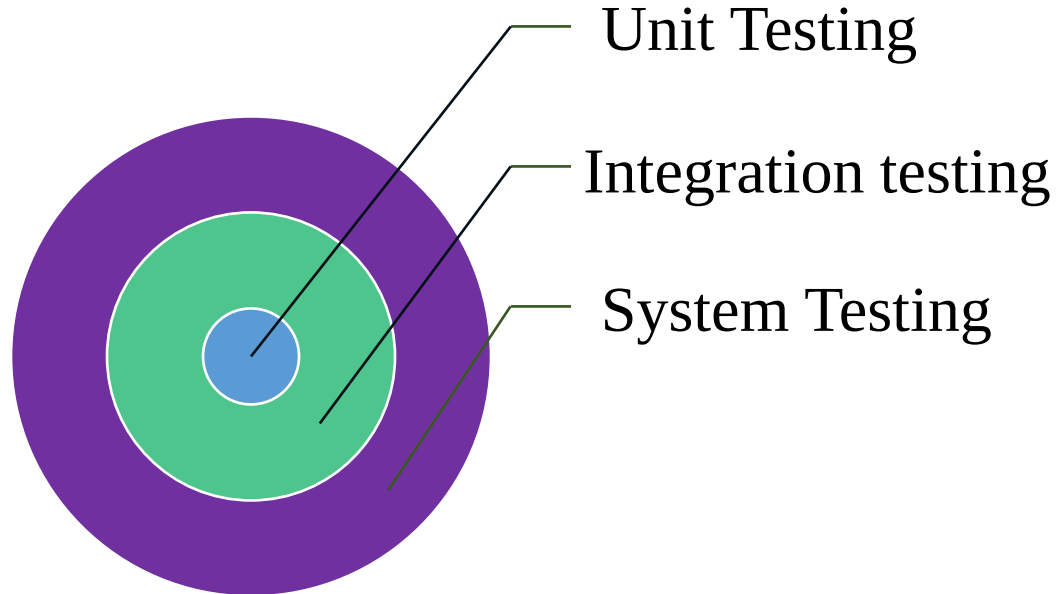
Test Execution: Testers execute the software based on the plans and tests and report any errors found to the development team.

Test Reporting: Once testing is completed, testers generate metrics and make final reports on their test effort and whether or not the software tested is ready for release.

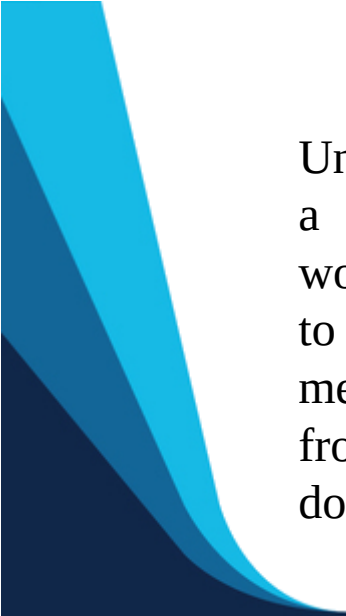
Retesting the Defects: Defects are once again tested to find whether they got eliminated or not.

Levels of Testing:

Mainly, Software goes through three levels of testing:



Unit Testing



Unit testing is a procedure used to verify that a particular segment of source code is working properly. The idea about unit tests is to write test cases for all functions or methods. Ideally, each test case is separate from the others. This type of testing is mostly done by developers and not by end users.

Unit Testing

The goal of unit testing is to isolate each part of the program and show that the individual parts are correct. Unit testing provides a strict, written contract that the piece of code must satisfy. Unit testing will not catch every error in the program. By definition, it only tests the functionality of the units themselves

Integration Testing

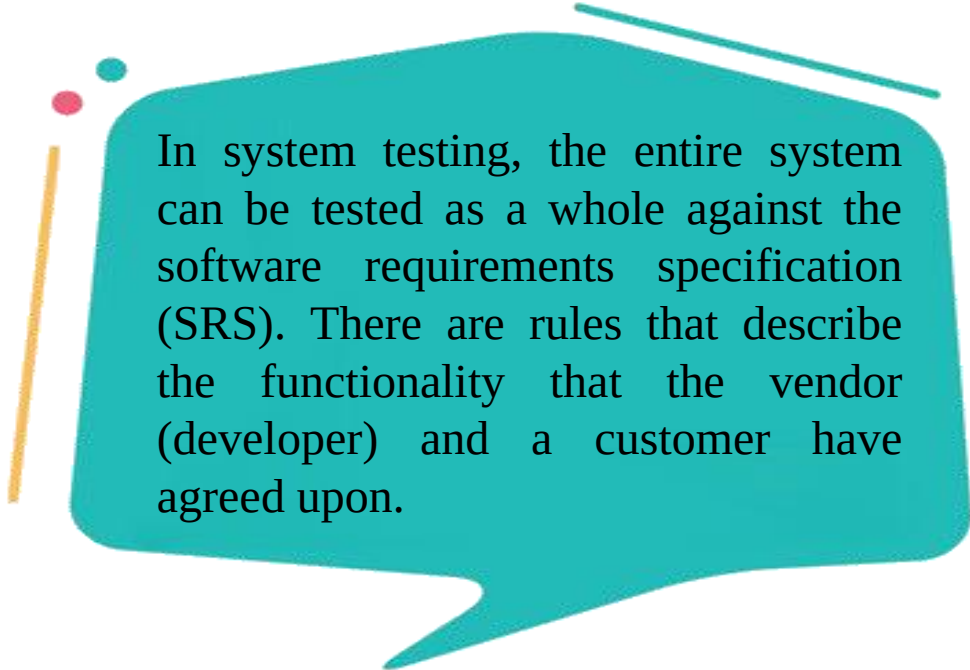
Integration testing (sometimes called **integration and testing**, abbreviated **I&T**) is the phase in software testing in which individual software modules are combined and tested as a group. It occurs after unit testing and before validation testing. Integration testing takes as its input modules that have been unit tested, groups them in larger aggregates, applies tests defined in an integration test plan to those aggregates, and delivers as its output the integrated system ready for system testing.

System Testing

System testing done by a professional testing agent on the completed software product before it is introduced to the market. System testing falls within the scope of black box testing, and as such, should require no knowledge of the inner design of the code or logic.

As a rule, system testing takes, as its input, all of the "integrated" software components that have passed integration testing and also the software system itself integrated with any applicable hardware system(s).

System Testing



In system testing, the entire system can be tested as a whole against the software requirements specification (SRS). There are rules that describe the functionality that the vendor (developer) and a customer have agreed upon.

Debugging

Debugging occurs as a consequence of successful testing. Debugging refers to the process of identifying the cause for defective behavior of a system and addressing that problem. In less complex terms - fixing a bug. When a test case uncovers an error, debugging is the process that results in the removal of the error. The debugging process begins with the execution of a test case. The debugging process attempts to match symptoms with cause, thereby leading to error correction. The following are two alternative outcomes of the debugging:

1. The cause will be found and necessary action such as correction or removal will be taken.
2. The cause will not be found..

Life Cycle of a Debugging Task

A. Defect Identification/Confirmation

- A problem is identified in a system and a defect report created
 - Defect assigned to a software engineer
 - The engineer analyzes the defect report, performing the following actions:
- What is the expected/desired behavior of the system? What is the actual behavior?
 - Is this really a defect in the system?
 - Can the defect be reproduced? (While many times, confirming a defect is straight forward. There will be defects that often exhibit quantum behavior.)

Life Cycle of a Debugging Task

B. Defect Analysis

Assuming that the software engineer concludes that the defect is genuine, the focus shifts to understanding the root cause of the problem. This is often the most challenging step in any debugging task, particularly when the software engineer is debugging complex software.

C. Defect Resolution

Once the root cause of a problem is identified, the defect can then be resolved by making an appropriate change to the system, which fixes the root cause.

Testing Tools

The following are different categories of tools that can be used for testing:

- ✓ **Data Acquisition:** Tools that acquire data to be used during testing.
- ✓ **Static Measurement:** Tools that analyze source code without executing test cases.
- ✓ **Dynamic Measurement:** Tools that analyze source code during execution.

Testing Tools

- ✓ **Simulation:** Tools that simulate functions of hardware or other externals.
- ✓ **Test Management:** Tools that assist in planning, development and control of testing.
- ✓ **Cross-Functional tools:** Tools that cross the bounds of preceding categories.

THANK YOU

Software Engineering

BLOCK 2: Software Project Management

MCS-213

Block-2

Unit-8 [Software change management]

9319133134 query@cftedu.in

Topics to be covered

What is Change Management

Software Configuration

Elements of a Configuration Management System

Baselines

Version Control

Change Control

Auditing and Reporting

What is Change Management

- Also called software configuration management (SCM)
- It is an umbrella activity that is applied throughout the software process.
- It's goal is to maximize productivity by minimizing mistakes caused by confusion when coordinating for software development.

What is Change Management

- SCM identifies, organizes, and controls modifications to the software being built by a software development team.
- SCM activities are formulated to identify change, control change and report changes to others who may have an interest
- SCM is initiated when the project begins and terminates when the software is taken out of operation.

What is Change Management

View of SCM from various roles

- Project manager : An auditing mechanism set by Project manager
- SCM manager : A controlling, tracking, and policy making method.
- Software Engineer : A change builder and access control mechanism is set by software engineer.
- Customer : Quality assurance product identification method is set by customer.

Software Configuration

☞ The Output from the software process makes up the software configuration

- Computer programs (both source code files and executable files)
- Work products that describe the computer programs (documents targeted at both technical practitioners and users)
- Data (contained within the programs themselves or in external files)

Software Configuration

☞ The major danger to a software configuration is change

- First Law of System Engineering: "No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle"

Origins of Software Change

- Errors detected in the software need to be corrected
- New business or market conditions dictate changes in product requirements or business rules
- New customer needs demand modifications of data produced by information systems, functionality delivered by products, or services delivered by a computer-based system
- Reorganization or business growth/downsizing causes changes in project priorities or software engineering team structure
- Budgetary or scheduling constraints cause a redefinition of the system

Elements of a Configuration Management System

Configuration elements : A set of tools coupled with a file mgmt. (e.g., database) system that enables access to and management of each software configuration item.

Process elements : A collection of procedures and tasks that define an effective approach to change management for all participants.

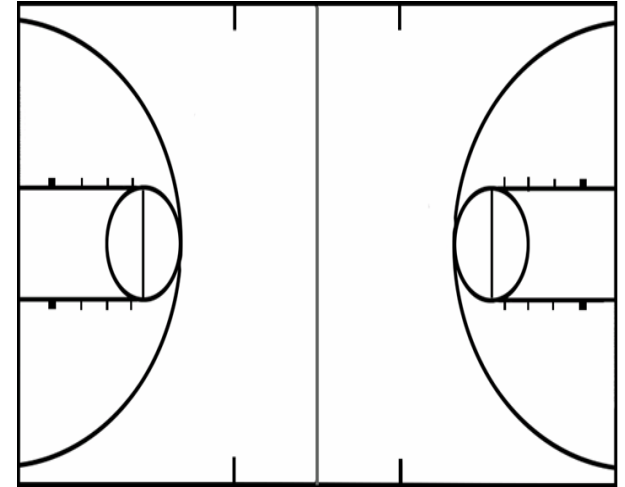
Elements of a Configuration Management System

Construction elements : A set of tools that automate the construction of software by ensuring that the proper set of valid components (i.e., the correct version) is assembled.

Human elements : A set of tools and process features used by software

Baseline

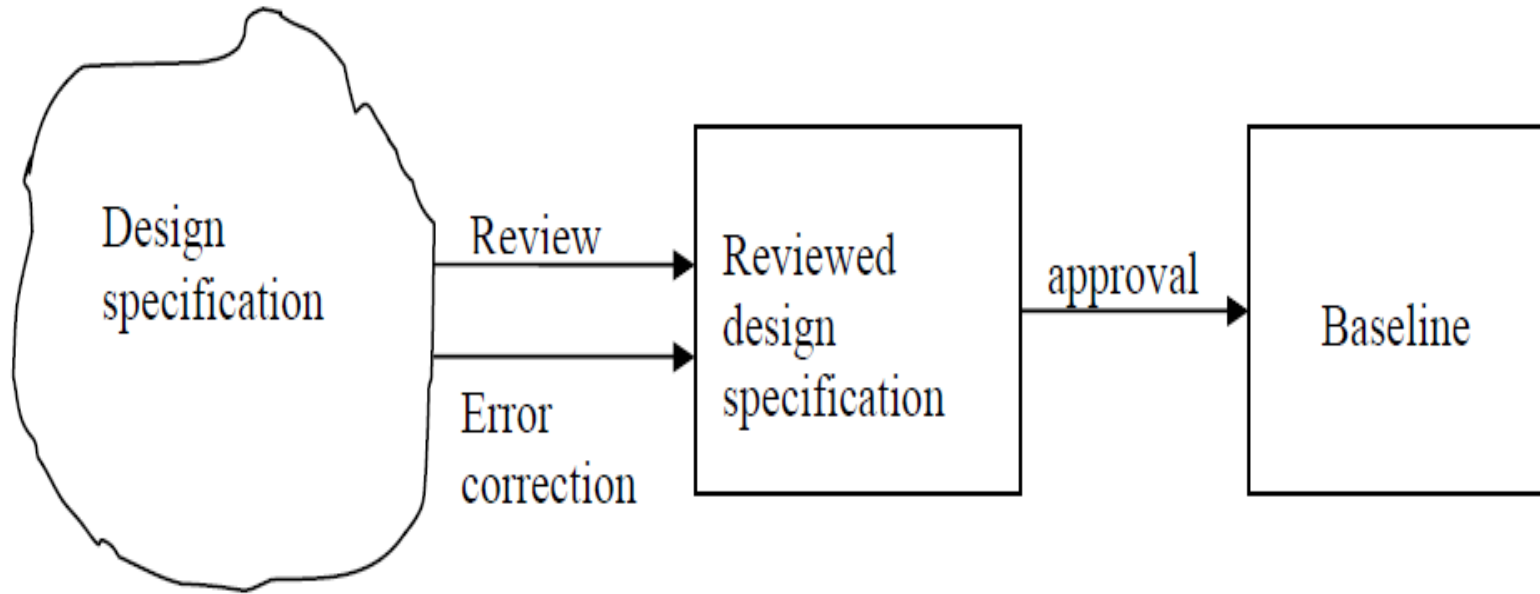
- An SCM concept that helps practitioners to control change without seriously impeding justifiable change
- IEEE Definition: A specification or product that has been formally reviewed and agreed upon, and that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures
- It is a milestone in the development of software and is marked by the delivery of one or more computer software configuration items (CSCIs) that have been approved as a consequence of a formal technical review



Base lining Process

- 1) A series of software engineering tasks produces a CSCI
- 2) The CSCI is reviewed and possibly approved
- 3) The approved CSCI is given a new version number and placed in a project database (i.e., software repository)
- 4) A copy of the CSCI is taken from the project database and examined/modified by a software engineer
- 5) The base lining of the modified CSCI goes back to Step #2

Base lining Process



SCM Repositories: Paper-based vs. Automated Repositories

❑ Problems with paper-based repositories (i.e. file cabinet containing folders)

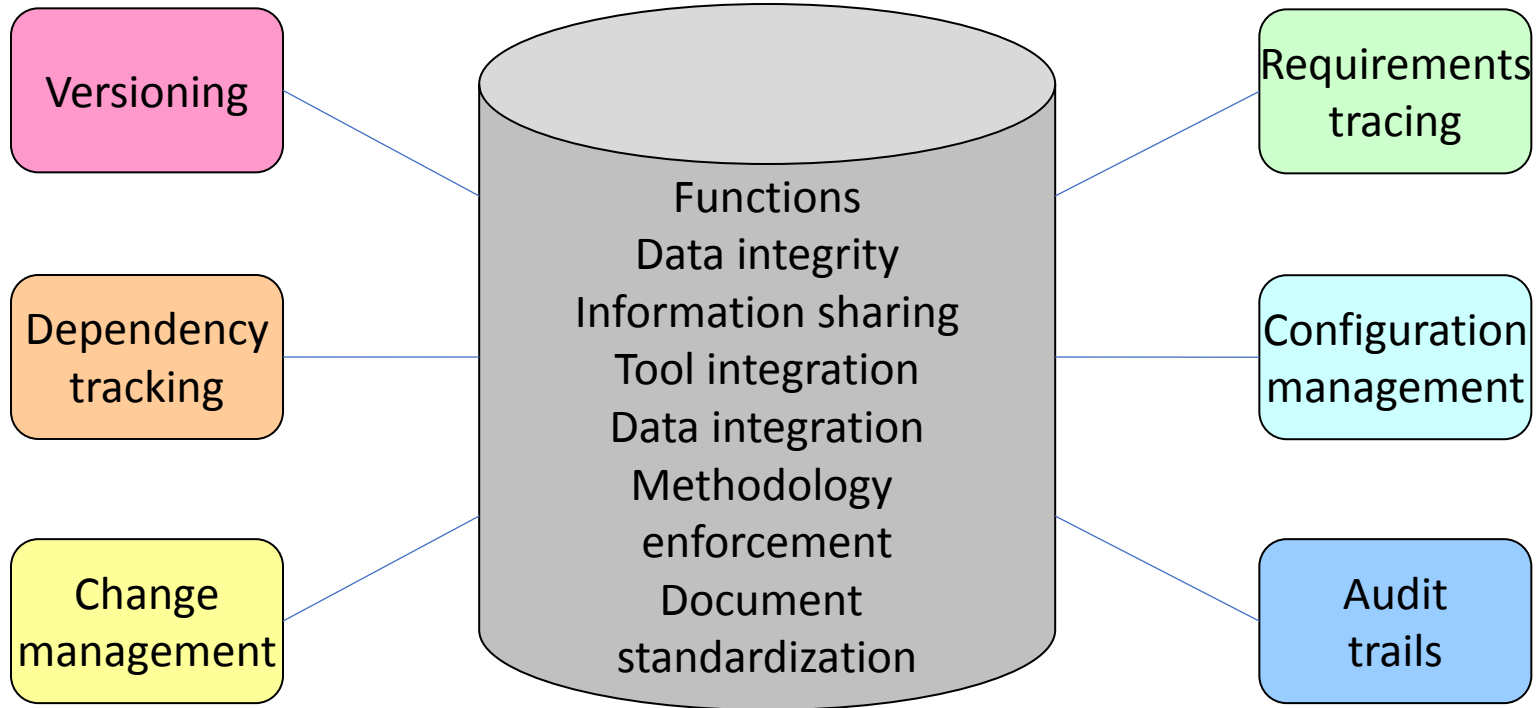
- Finding a configuration item when it was needed was often difficult
- Determining which items were changed, when and by whom was often challenging
- Constructing a new version of an existing program was time consuming and error prone
- Describing detailed or complex relationships between configuration items was virtually impossible

SCM Repositories: Paper-based vs. Automated Repositories

Today's automated SCM repository

- It is a set of mechanisms and data structures that allow a software team to manage change in an effective manner
- It acts as the center for both accumulation and storage of software engineering information
- Software engineers use tools integrated with the repository to interact with it

Automated SCM Repository (Functions and Tools)



Functions of an SCM Repository

- **Data integrity**
 - Validates entries, ensures consistency, cascades modifications
- **Information sharing**
 - Shares information among developers and tools, manages and controls multi-user access
- **Tool integration**
 - Establishes a data model that can be accessed by many software engineering tools, controls access to the data

Functions of an SCM Repository

Data integration

Allows various SCM tasks to be performed on one or more CSCIs

Methodology enforcement

Defines an entity-relationship model for the repository that implies a specific process model for software engineering

Toolset Used on a Repository

Document standardization

- Defines objects in the repository to guarantee a standard approach for creation of software engineering documents

Versioning

- Save and retrieve all repository objects based on version number

Dependency tracking and change management

- Track and respond to the changes in the state and relationship of all objects in the repository

Toolset Used on a Repository

Dependency tracking and change management

- Track and respond to the changes in the state and relationship of all objects in the repository

Requirements tracing

- (Forward tracing) Track the design and construction components and deliverables that result from a specific requirements specification
- (Backward tracing) Identify which requirement generated any given work product

Configuration management

Track a series of configurations representing specific project milestones or production releases

Objectives of Software Change Management Process

- 1) **Configuration identification:** The process of identification involves identifying each component name, giving them a version name (a unique number for identification) and a configuration identification.
- 2) **Configuration control:** Controlling changes to a product. Controlling release of a product and changes that ensure that the software is consistent on the basis of a baseline product.

Objectives of Software Change Management Process

3) Review: Reviewing is the process to ensure consistency among different configuration items.

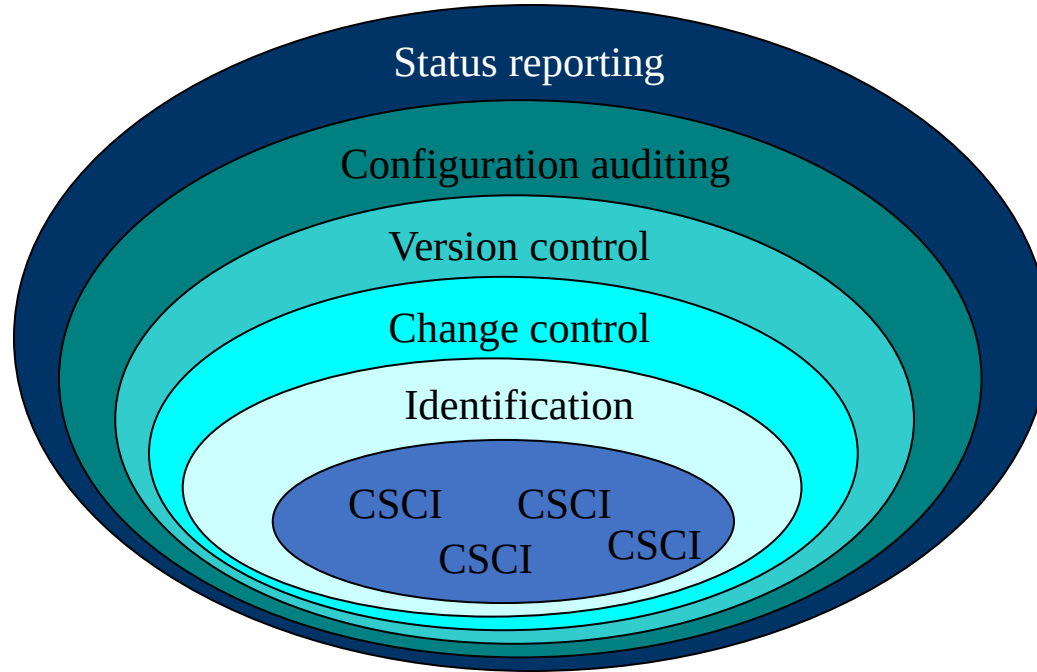
4) Status accounting : Recording and reporting all the changes and status of the components.

5) Auditing and reporting: Validating the product and maintaining product.

SCM Questions

- How does a software team identify the discrete elements of a software configuration?
- How does an organization manage the many existing versions of a program (and its documentation) in a manner that will enable change to be accommodated efficiently?
- How does an organization control changes before and after software is released to a customer?
- Who has responsibility for approving and ranking changes? How can we ensure that changes have been made properly? What mechanism is used to appraise others of changes that are made?

SCM Tasks



SCM Tasks

- Identification separately names each CSCI and then organizes it in the SCM repository using an object-oriented approach
- Objects start out as basic objects and are then grouped into aggregate objects

SCM Tasks

- Each object has a set of distinct features that identify it
 - A name that is unambiguous to all other objects
 - A description that contains the CSCI type, a project identifier, and change and/or version information. List of resources needed by the object i.e. document, file, Model, etc.

Change Control Task

- Change control is a procedural activity that ensures quality and consistency as changes are made to a configuration object
- A change request is submitted to a configuration control authority, which is usually a change control board (CCB)
 - The request is evaluated for technical merit, potential side effects, overall impact on other configuration objects and system functions, and projected cost in terms of money, time , resources.

Change Control Task

- An engineering change order (ECO) is issued for each approved change request . It describes the change to be made, the constraints to follow, and the criteria for review and audit.
- The baseline CSCI is obtained from the SCM repository. Access control governs which software engineers have the authority to access and modify a particular configuration object. Synchronization control helps to ensure that parallel changes performed by two different people don't overwrite one another.

Version Control Task

Version control is a set of procedures and tools for managing the creation and use of multiple occurrences of objects in the SCM repository **Required version control capabilities**

- An SCM repository that stores all relevant configuration objects
- A version management capability that stores all versions of a configuration object (or enables any version to be constructed using differences from past versions)
- Version Control enables the software engineer to collect all relevant configuration objects and construct a specific version of the software

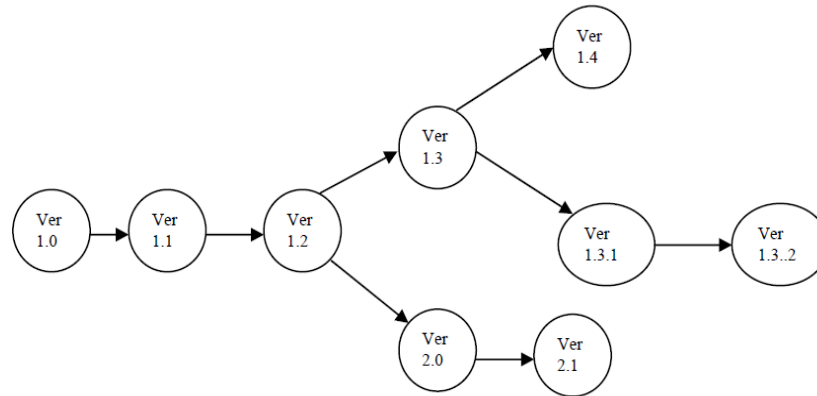
Version Control Task

- Issues tracking (bug tracking) capability that enables the team to record and track the status of all outstanding issues associated with each configuration object.

The SCM repository maintains a change set

- Serves as a collection of all changes made to a baseline configuration
- Used to create a specific version of the software
- Captures all changes to all files in the configuration along with the reason for changes and details of who made the changes and when.

Version Control Task



An evolutionary graph for a different version of an item

Configuration Auditing Task

- Configuration auditing is an SQA activity that helps to ensure that quality is maintained as changes are made
- It complements the formal technical review and is conducted by the SQA group

Configuration Auditing Task

It addresses the following questions

- Has the change specified in the ECO been made? Have any additional modifications been incorporated?
- Has a formal technical review been conducted to assess technical correctness?
- Has the software process been followed, and have software engineering standards been properly applied?

Configuration Auditing Task

- ☐ Has the change been "highlighted" and "documented" in the CSCI?
- ☐ Have the change data and change author been specified? Do the attributes of the configuration object reflect the change?
- ☐ Have SCM procedures for noting the change, recording it, and reporting it been followed? Have all related CSCIs been properly updated?

Configuration Auditing Task

- ❑ A configuration audit ensures that
The correct CSCIs (by version) have
been incorporated into a specific
build That all documentation is up-
to-date and consistent with the
version that has been built

Status Reporting Task

Configuration status reporting (CSR) is also called status accounting
Provides information about each change to those personnel in an organization with a need to know

Answers what happened, who did it, when did it happen, and what else will be affected?

Sources of entries for configuration status reporting

- Each time a CSCI is assigned new or updated information
- Each time a change is approved by the CCB and an ECO is issued
- Each time a configuration audit is conducted

Status Reporting Task

- The configuration status report
 - Placed in an on-line database or on a website for software developers and maintainers to read
 - Given to management and practitioners to keep them appraised of important changes to the project CSCIs

THANK YOU

Software Engineering

BLOCK 3: Web, Mobile and CASE tools
MCS-213
Block-3

Unit-9 [Web Software Engineering]

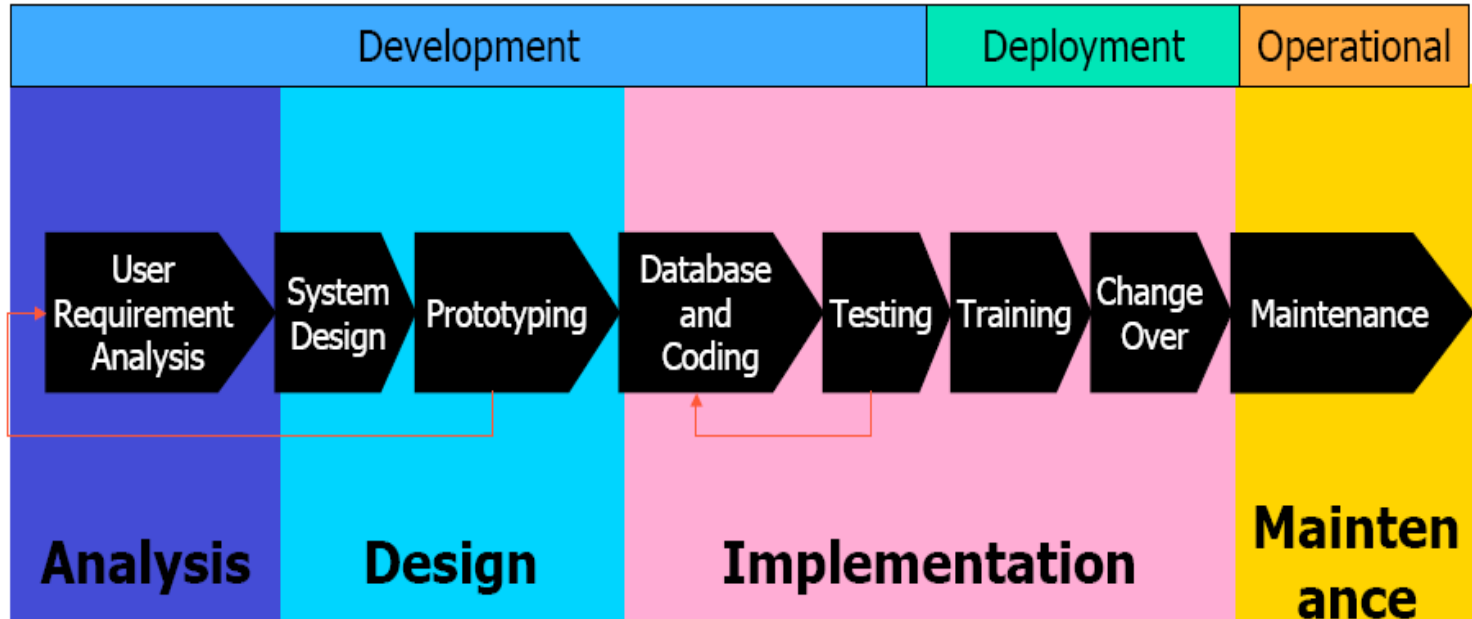
Topics to be Covered

- Characteristics of a Web Application
- Development, Testing and Deployment
- Maintaining Web Applications
- Designing Web Applications
- Issues of Management of Web Based Projects
 - Organization of Web Application Teams
 - Development and Maintenance Issues
- Metrics
- Analysis
- Design and Construction
- Reviews and Testing

Web Engineering

- ❖ **Web engineering** is body of knowledge consisting of Technologies, Architectures, Design Methodologies and Development Processes, that will enable us to develop Complex and Maintainable Web Systems.
- ❖ Web engineering based on Software Engineering, Web Engineering comprises the use of systematic and quantifiable approaches in order to accomplish the specification, implementation, operation, and maintenance of high quality Web applications

Life Cycle



Characteristics Of A Web Application

There can be many kinds of web applications such as

- Those with static text
- Content that is changed frequently
- Interactive websites that act on user input
- Portals that are merely gateways to different kinds of websites
- Commercial sites that allow transactions
- Those that allow searches.

Development, Testing and Deployment

- Development of a Web site is not an event. It is a process. Once developed, information in the Web site needs to be maintained.
- Testing is the process of evaluating a system or its component(s) with the intent to find whether it satisfies the specified requirements or not. Testing is executing a system in order to identify any gaps, errors, or missing requirements in contrary to the actual requirements.

Development, Testing and Deployment

- Software deployment is all of the activities that make a software system available for use. The general deployment process consists of several interrelated activities with possible transitions between them. These activities can occur at the producer side or at the consumer side or both.

Maintaining Web Applications

Once your Web application has been deployed, you can easily add or update individual pieces of the application. You might want to update your Web application with new pages, images, or other functionality. Or perhaps you made changes to files that are already deployed. You can copy over the currently deployed files with new versions.

Maintaining Web Applications

Typically, even while the application files are browsed by an end user, you can make changes without affecting the user. Users are protected because a temporary copy of the file was downloaded to the user when it was requested. The new version is available when the user refreshes the file in the Web browser

Designing Web Applications

❖ When designing a Web application, the goal of the software architect is to minimize the complexity by separating tasks into different areas of concern while designing a secure, high performance application.

❖ Web Design and Applications involve the standards for building and Rendering Web pages, including HTML, CSS, SVG, device APIs, and other technologies for Web Applications

Designing Web Applications

- ❖ When designing a Web application, the goal of the software architect is to minimize the complexity by separating tasks into different areas of concern while designing a secure, high performance application.
- ❖ Web Design and Applications involve the standards for building and Rendering Web pages, including HTML, CSS, SVG, device APIs, and other technologies for Web Applications.

Mobile web applications

- ❖ **Mobile web applications** are accessed through a mobile device's browser, and rely on web access. So while Facebook has a mobile application (the icon shortcut), it has a mobile web application (essentially a website designed specifically for mobile) when it's opened in the browser. Some websites are designed and enhanced to meet mobile needs.
- ❖ **Web applications** use the web and browser capacities to accomplish honed task(s.)

Continue...

- ❖ Differentiating web application or apps from website boundaries are dynamic. The important takeaway is apps are generally small, and do something specific.
- ❖ Some photography sites are web applications (think Flickr's website on your mobile browser, LinkedIn on your desktop or Yahoo! Mail on your tablet browser,) whereas a company's marketing page is a website. Such sites can be accessed by mobile device browsers, but only if they have web access.

Continue...

- ❖ Examples of web applications include browser add-ons, online retail stores, email platforms and Flash games.
- ❖ **Desktop applications** run on a desktop, and don't need web access to function. They could be represented by icons and often come standard with new computers. Examples include Paint, Notepad and iPhoto. They could also refer a custom “application” used for a specific purpose within a corporate environment.

Continue...

This chart can help clear up differences:

	Desktop	Web	Mobile	Mobile Web
Facebook	None	Open through browser on computer (Chrome, Safari, FireFox, IE, etc)	Open through icon downloaded in Google Play, iTunes, etc.	Open through browser on device at http://m.facebook.com
Photo Editing	iPhoto, Paint, Microsoft Office Picture Manager	Sites like Flickr and PicMonkey—launched by computer	Instant Retro Photo, PicSay (Android); PicStitch, Be Funky Photo Editor (Apple)	Flickr, etc, launched through mobile browser
Solitaire	Comes stock on Microsoft in “Accessories”—doesn’t need Internet	worldofsolitaire.com; games.com; solitaire-cardgame.com	The Solitaire Games (Apple); Solitaire Free Pack (Android)	worldofsolitaire.com; games.com; solitaire-cardgame.com opened in mobile browser

This project is aimed at :

- 1) Research of MSE-focused programming methodologies
- 2) Analysis models in MSE
- 3) Design and development models in MSE, including architectural models, information models, functional models, interaction models, navigation models, graphic user interface (GUI) hierarchical models,.
- 4) Analysis of integrated development environments (IDEs) for various mobile platforms (Android, Windows Phone, etc.).
- 5) Testing strategies and techniques for mobile software systems
- 6) Mobile software quality management
- 7) Security issues of mobile software systems
- 8) MSE-focused implementation methods.

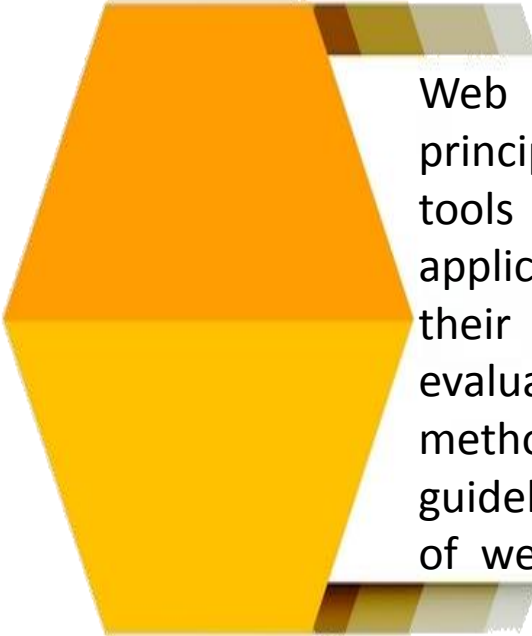
What is Web Software Engineering

- ❖ Web engineering is multidisciplinary and encompasses contributions from diverse areas: System Analysis and Design, software engineering, hypertext engineering, requirements engineering, human-computer interaction, user interface, information engineering, information indexing and retrieval, testing, modeling and simulation, project management, and graphic design and presentation.

What is Web Software Engineering

- ❖ Web engineering is neither a clone nor a subset of software engineering, although both involve programming and software development. It uses software engineering principles, methodologies, techniques, and tools that are the foundation of Web application development and which support their design, development, evolution, and evaluation. It encompasses new approaches, methodologies, tools, techniques and guidelines to meet the unique requirements of web-based applications

What is Web Software Engineering Continue...



Web Engineering uses software engineering principles, methodologies, techniques, and tools that are the foundation of Web application development and which support their design, development, evolution, and evaluation. It encompasses new approaches, methodologies, tools, techniques, and guidelines to meet the unique requirements of web-based applications.

Issues Of Mgt. Of Web Based Projects

Managing Requirements

While a requirements document might be prepared and approved by the customer, there are many issues that are hard to capture in any document, despite having detailed discussion with customer. To ensure that the viewpoints of the developers and customer do not diverge, regular communication is a must.

Managing the Project

The essence of managing the project is proper planning for the project and then executing the project according to the plan. If the project deviates from the plan, we need to take corrective action or change the plan.

Issues Of Mgt. Of Web Based Projects

Besides the project management plan, there are various subordinate plans that need to be made for different aspects of the project.

Configuration Management Plan

Quality Assurance Plan that covers audits and process compliance

Quality Control Plan that covers technical reviews and testing

Risk Management Plan

Training Plan

Metrics Plan

Defect Prevention Plan

Issues Of Mgt. Of Web Based Projects

Configuration Management

Software configuration management (SCM) is the task of tracking and controlling changes in the software, part of the larger cross-discipline field of configuration management. SCM practices include revision control and the establishment of baselines.

Issues Of Mgt. Of Web Based Projects



❖ Measurements

It is important to measure our software work so that we can keep control of the proceedings. This is the key to being able to manage the project. Without measurements we will not be able to objectively assess the state of the progress and other parameters of the project.

❖ Risk Management

A risk is an event that can have a deleterious impact on the project, but which may or may not occur. Thinking of the possible risks is the first step in trying to take care of them



Organization of Web Application Teams

Webmaster

It entails taking care of the site on a day to day basis and keeping it in good health. It involves close interaction with the support team to ensure that problems are resolved and the appropriate changes made.

Gathering user feedback, both on bugs (such as broken links) and also on suggestions and questions from the public. These need to be passed on to the support team who are engaged in adapting the site in tune with this information.

Ensuring proper access control and security for the site. This could include authentication, taking care of the machines that house the server and so on.

Organization of Web Application Teams

❖ **Application Support Team :**

- Removing bugs and taking care of cosmetic irritants.
- Changing the user interface from time to time for novelty, to accommodate user feedback and to make it more contemporary.
- Altering the business rules of the application as required.
- Introducing new features and schemes, while disabling or removing older ones.

Organization of Web Application Teams



➤ **Content Development Team**

Web applications frequently require much more ongoing, sustained effort to retain user interest because of the need to change the content in the site. Content development teams could be researchers who seek out useful or interesting facts, authors, experts in different areas, and so on. The content may be edited before being served out.

➤ **Web Publisher**

This is an important role that connects the content development team to the actual website. The raw material created by the writers has to be converted into a form that is suitable for serving out of a webserver.



Development and Maintenance Issues



Configuration Management



Project Management

Metrics

- Metric - Quantitative measure of degree to which a system, component or process possesses a given attribute. “A handle or guess about a given attribute.”
- Effort metrics can be gathered through a time sheet and we can measure the total effort in different kinds of activities to build up our metrics database for future guidance.
- The quality of the application can be measured in terms of defects that are found during internal testing or reviews as well as external defects that are reported by visitors who come to the site.

Analysis

Requirements analysis in systems engineering and software engineering, encompasses those tasks that go into determining the needs or conditions to meet for a new or altered product, taking account of the possibly conflicting requirements of the various stakeholders, analyzing, documenting, validating and managing software or system requirements.

Design And Construction

Where we have to lay great emphasis is on the user interface. There are several aspects to creating a good one, some of which are:

- Visual appeal and aesthetics
- Ease of entry of data
- Ease of obtaining the application's response
- Robust and forgiving of errors

Reviews And Testing

- While in a conventional application a program is the smallest unit that we test, in web applications it is more likely to be a single web page. We should check out the content, the navigation and the links to other pages as part of testing out a page.
- Once all the pages of the web application are ready we can perform integration testing. We check out how the pages work together.
- Whenever an error is caught and fixed we should repeat the tests to ensure that solving a problem has not caused another problem elsewhere.

THANK YOU

Software Engineering

BLOCK 3: Web, Mobile and CASE tools

MCS-213

Block-3

Unit-10 [Mobile Software Engineering]

Topics to be Covered

- Formal Methods
- Limitations of Formal Specification Using Formal Methods
- Cleanroom Software Engineering
- Cleanroom Software Engineering Principles, Strategy and Process Overview
- Limitations of Cleanroom Engineering
- Software Reuse and its Types
- Component based Software Engineering (CBSE) Process
- Reengineering: An Introduction
- Software Reengineering Life Cycle

Formal Methods

A **Formal method** in software development is a method that provides a formal language for describing a software artifact (e.g., specifications, designs, source code) such that formal proofs are possible, in principle, about properties of the artifact so express

Formal Methods

The major concerns of the formal methods are as follows:

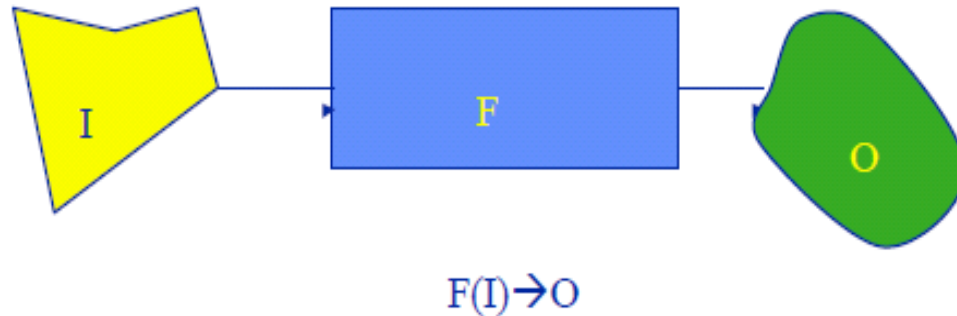
- i) The correctness of the problem
 - producing software that is “correct” is famously difficult;
 - by using rigorous mathematical techniques, it may be possible to make probably correct software

Formal Methods

- ii) Programs are mathematical objects
 - they are expressed in a **formal** language;
 - they have a **formal** semantics;
 - programs can be treated as mathematical theories

Use Of Mathematics In Software Development

- $\text{Program} \Leftrightarrow \text{Mathematical Object}$
- $\text{Programming Language} \Leftrightarrow \text{Mathematical Language}$
- Can prove properties about the program



Goals of Formal Specification

Removal of ambiguity

Consistency

Completeness

Application Areas

The following are the Formal specifications application areas:

Safety critical systems

Security systems

The definition of standards

Hardware development

Operating systems

Transaction processing systems

Anything that is hard, complex, or critical

Limitations Of Using Formal Methods

- Cost:** It can be almost as costly to do a full formal specification as to do all the coding.
- Complexity:** Not everybody can read formal specifications (especially customers and users).
- Deficiencies of Less Formal Approaches:** The methods of structured and object-oriented design discussed previously make heavy use of natural language and a variety of graphical notations.

Thank You!

Software Engineering

BLOCK 3: Web, Mobile and CASE tools

MCS-213
Block-3

Unit-11 [CASE Tools]

Topics to be Covered

What are CASE Tools

Categories of CASE Tools

Need of CASE Tools

Characteristics of a successful CASE Tool

CASE Tools and Requirement Engineering

CASE Tools and Design and Implementation

Software Testing

Software Quality and CASE Tools

Software Project Management and CASE Tools

CASE Tools

CASE tools are the software engineering tools that permit collaborative software development and maintenance. CASE tools support almost all the phases of the software development life cycle such as analysis, design, etc., including umbrella activities such as project management, configuration

CASE Tools

CASE tools may support the following development steps:

- Creation of data flow and entity models
- Establishing a relationship between requirements and models
- Development of top-level design
- Development of functional and process description
- Development of test cases.

Categories Of CASE Tools

CASE tools are classified into the following categories:

1. Upper CASE tools

Upper CASE tools mainly focus on the analysis and design phases of software development.

2. Lower CASE tools

Lower CASE tools support implementation of system development. They include tools for coding, configuration management, etc.

3. Integrated CASE tools

Integrated CASE tools help in providing linkages between the lower and upper CASE tools.

Need of CASE Tools

The CASE tools provide the integrated homogenous environment for the development of complex projects. They allow creating a shared repository of information that can be utilized to minimize the software development time.

Need of CASE Tools

- Reduce the cost as they automate many repetitive manual tasks.
- Reduce development time of the project as they support standardization and avoid repetition and reuse.
- Develop better quality complex projects as they provide greater consistency and coordination.
- Create good quality documentation
- Create systems that are maintainable because of proper control of configuration item that
- support traceability requirements.

Characteristics of CASE Tools

A standard methodology

Flexibility

Strong Integration

Integration with testing software

Support for reverse engineering

On-line help

Case Software Development Environment

CASE tools support extensive activities during the software development process. Some of the functional features that are provided by CASE tools for the software development process are:

Case Software Development Environment

Creating software requirements specifications

Creation of design specifications

Creation of cross references.

Verifying/Analyzing the relationship between requirement and design

Performing project and configuration management

Building system prototypes

Containing code and accompanying documents

Validation and verification, interfacing with external environment

Case Tools And Requirement Engineering

A good and effective requirements engineering tool needs to incorporate the best practices of requirements definition and management.

- CASE tool must have the following features from the requirements engineering viewpoint:
- A dynamic, rich editing environment for team members to capture and manage requirements
- To create a centralized repository .
- To create task-driven workflow to do change management, and defect tracking.

Case Tools And Requirement Engineering

Validation of Requirements

- Check that the right product is being built
- Ensures that the software being developed (or changed) will satisfy its stakeholders.
- Checks the software requirements specification against stakeholders goals and requirements

Managing Requirements

- Estimation Of Efforts And Cost
- Specification Of Project Schedule Such As Deadline, Staff Requirements And Other Constraints
- Specification Of Quality Parameters.

Case Tools And Design And Implementation

CASE tools support the analysis and design phases of software development. Some of the tools supported by the design tools are:

Structured Chart

Program Document Language (PDL).

Optimization of ER and other models

Flow charts

Database design tools

File design tools

CASE Repository

CASE Repository stores software system information. It includes analysis and design specifications and helps in analyzing these requirements and converting them into program code and documentation.

CASE Repository

Data: Information and entities/object relationship attributes, etc.

Process: Support structured methodology, link to external entities, document structured software development process standards.

Models: Flow models, state models, data models, UML document etc.

Rules/Constraints: Business rules, consistency constraints, legal constraints.

Software Testing

Features Needed for Testing

- The tool must support all testing phases, viz., plan, manage and execute all types of tests viz., functional, performance, integration, regression, testing from the same interface.
- It should integrate with other third party testing tools.
- It should support local and remote test execution.
- It should create a log of test run.
- It should output meaningful reports on test completeness, test analysis.
- It may provide automated testing

Software Configuration Management

Software Configuration Management (SCM) is extremely important from the view of deployment of software applications. SCM controls deployment of new software version. SCM should have the following facilities:

Software Configuration Management

- Creation of configuration
- This documents a software build and enables versions to be reproduced on demand
- Configuration lookup scheme that enables only the changed files to be rebuilt. Thus, entire application need not be rebuilt.
- Dependency detection features even hidden dependencies, thus ensuring correct behavior of the software in partial rebuilding.

Software Project Management And Case Tools



The CASE tools help in effective management of teams and projects. Let us look into some of the features of CASE in this respect:

Software Project Management And Case Tools

- Sharing and securing the project using the user name and passwords.
- Allowing reading of project related documents
- Allowing exclusive editing of documents
- Linking the documents for sharing
- Automatically communicative change requests to approver and all the persons who are sharing the document.
- It should support drawing of schedules using PERT and Gantt chart.
- It should be easy to use such that tasks can be entered easily, the links among the tasks should be easily desirable.
- Milestones and critical path should be highlighted.

Thank
you

Software Engineering

MCS-213
Block-3

BLOCK 3: Web, Mobile and CASE tools

Unit-12 [Advanced Software Engineering]

Topics to be Covered

Cleanroom Software Engineering

Cleanroom Software Engineering Principles, Strategy and Process Overview

Limitations of Cleanroom Engineering

Software Reuse and its Types

Component based Software Engineering (CBSE) Process

Reengineering: An Introduction

Cleanroom Software Engineering

Cleanroom software engineering is an engineering and managerial process for the development of high-quality software with certified reliability. Cleanroom was originally developed by Dr. Harlan Mills. The name “Cleanroom” was taken from the electronics industry, where a physical clean room exists to prevent introduction of defects during hardware fabrication.

Principles For The Cleanroom-based Software Development

Incremental development under statistical quality control (SQC): Incremental development as practiced in Cleanroom provides a basis for statistical quality control of the development process.

Principles For The Cleanroom-based Software Development

Software development based on mathematical principles: In Cleanroom software engineering development, the key principle is that, a computer program is an expression of a mathematical function

Software testing based on statistical principles: In Cleanroom, software testing is viewed as a statistical experiment.

Cleanroom Software Engineering Principles, Strategy And Process Overview continue...

Increment planning: The project plan is built around the incremental strategy.

Requirements gathering: Customer requirements are elicited and refined for each increment using traditional methods.

Box structure specification: Box structures isolate and separate the definition of behavior, data, and procedures at each level of refinement.

Cleanroom Software Engineering Principles, Strategy And Process Overview continue...

Formal design: Specifications (black-boxes) are iteratively refined to become architectural designs (state-boxes) and component-level designs (clear boxes).

Correctness verification: Correctness questions are asked and answered, formal mathematical verification is used as required.

Cleanroom Software Engineering Principles, Strategy And Process Overview

Code generation, inspection, verification:

Box structures are translated into program language; inspections are used to ensure conformance of code and boxes.

Statistical test planning: A suite of test cases is created to match the probability distribution of the projected product usage pattern.

Statistical use testing: A statistical sample of all possible test cases is used rather than exhaustive testing.

Limitations Of Cleanroom Engineering

- Some people believe cleanroom techniques are too theoretical, too mathematical, and too radical for use in real software development.
- Relies on correctness verification and statistical quality control rather than unit testing (a major departure from traditional software development).
- Organizations operating at the ad hoc level of the Capability Maturity Model do not make rigorous use of the defined processes needed in all phases of the software lifecycle.

Software Reuse And Its Types

To achieve better software quality, more quickly, at lower costs, software engineers are beginning to adopt systematic reuse as a design process.

Application System Reuse

Component Reuse

Function Reuse

Component Based Software Engineering Process

CBSE is in many ways similar to conventional or object-oriented software engineering. A software team establishes requirements for the system to be built using conventional requirements elicitation techniques. An architectural design is established.

Two processes occur in parallel during the CBSE process. These are:

Domain Engineering

Component Based Development

Reengineering

The Process of transformation of a product with the vision of fulfilling the new business requirements along with the existing requirement is called Software Engineering.

Objectives Of Reengineering :

Improve quality

Migration

Reengineering

Reverse Engineering is a process of redesigning an existing product to improve and broaden its functions, add quality and to increase its useful life.

The main aim of reverse engineering is to reduce manufacturing costs of the new software product, making it competitive in market

Why Reverse Engineering

The original producer no longer produces the product.

As there is inadequate documentation of product.

Why Reverse Engineering

Some bad features of the product needs to be redesigned

To update obsolete materials with more current, less expensive technologies

Advantages of Reverse Engineering

- RE typically starts with measuring an existing object, so that a solid model can be deduced in order to make use of the advantages of CAD/CAM/CAE technologies.
- CAD models are used for manufacturing or rapid prototyping applications.

Advantages of Reverse Engineering

Hence we can work on a product without having prior knowledge of the technology involved. Cost saving for developing new products.

Lesser maintenance costs, Quality improvement. & Competitive

Applications in Manufacturing Field

- To create a 3D virtual model of an existing physical part for use in 3D CAD, CAM, CAE or other software and to analyze the working of a product.
- Medical Field: Imaging, modeling and replication (as a physical model) of a patient's bone structure
- Software engineering: To detect and neutralize viruses and malware.

Software Reengineering Life Cycle

Requirements analysis phase

Model analysis phase

Source code analysis phase

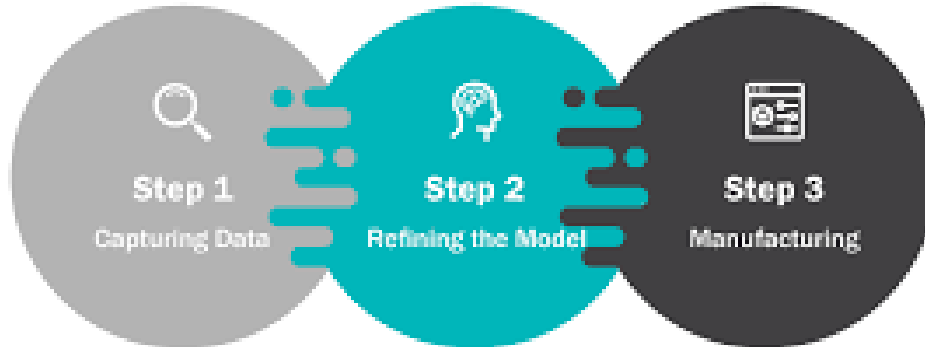
Remediation phase

Transformation phase

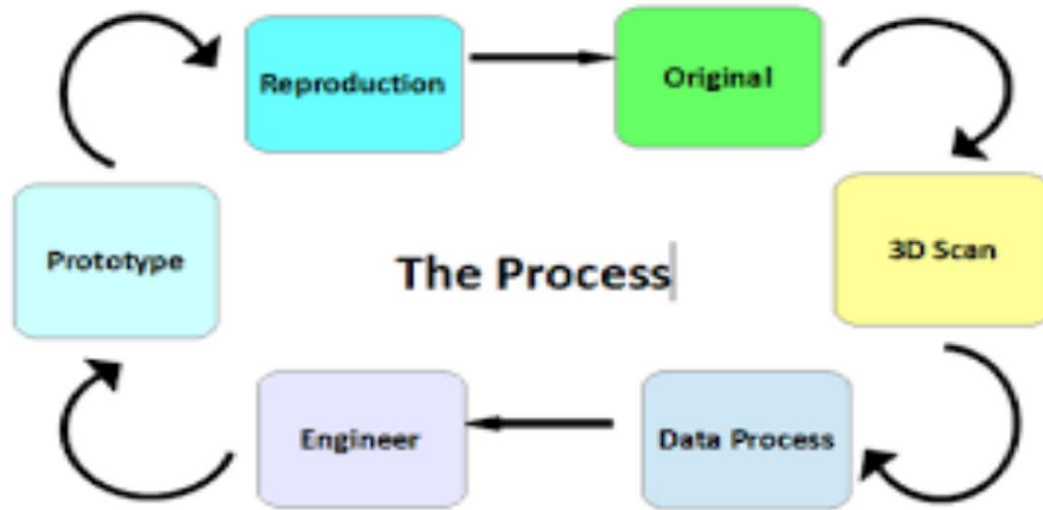
Evaluation phase

Reverse Engineering

3 Steps of Reverse Engineering



Reverse Engineering Process



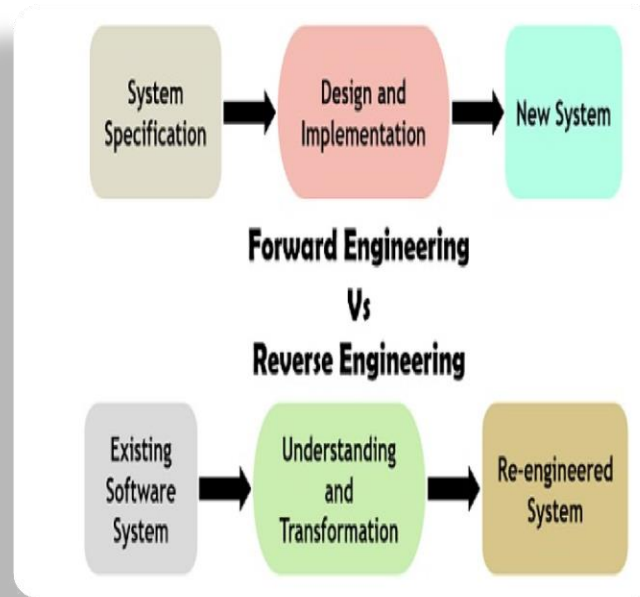
Reverse engineering

- Reverse engineering teaches the inner workings of any processor
- Learning how the processor handles data helps in understanding many other aspects of computer security



Comparison

BASIS FOR COMPARISON	FORWARD ENGINEERING	REVERSE ENGINEERING
Basic	Development of the application with provided requirements.	The requirements are deduced from the given application.
Certainty	Always produces an application implementing the requirements.	One can yield several ideas about the requirement from an implementation.
Nature	Prescriptive	Adaptive
Needed skills	High proficiency	Low-level expertise
Time required	More	Less
Accuracy	Model must be precise and complete.	Inexact model can also provide partial information.



Reverse engineering

- Reverse engineering is a vast and complex world
- With a lot of practice though it becomes much easier
- A good reverser knows their tools inside and out
- Workflow and organization are the keys to reversing

Resources

<https://slideplayer.com/slide/9960611/>

Thank you!

Software Engineering

BLOCK 4: Advanced Topics in Software Engineering

MCS-213

Block-4

Unit-13 [SPI - Software Process Improvement]

Software Process Improvement “Never Stop Learning”

Software Process Improvement



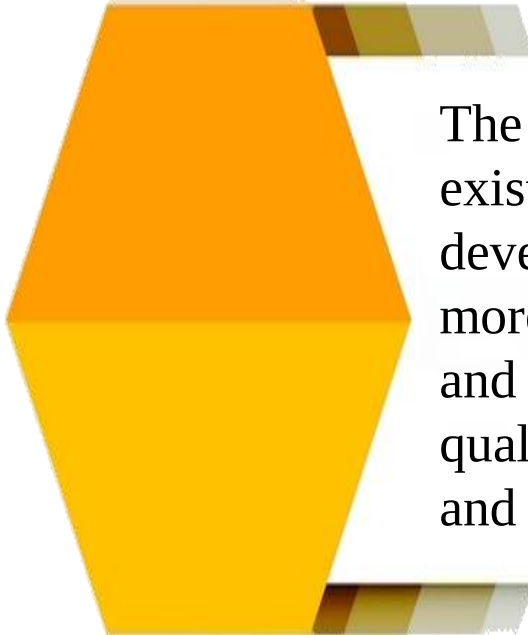
"Never Stop Learning"

What is SPI?

Software Process Improvement implies that elements of an effective software process can be defined in an effective manner

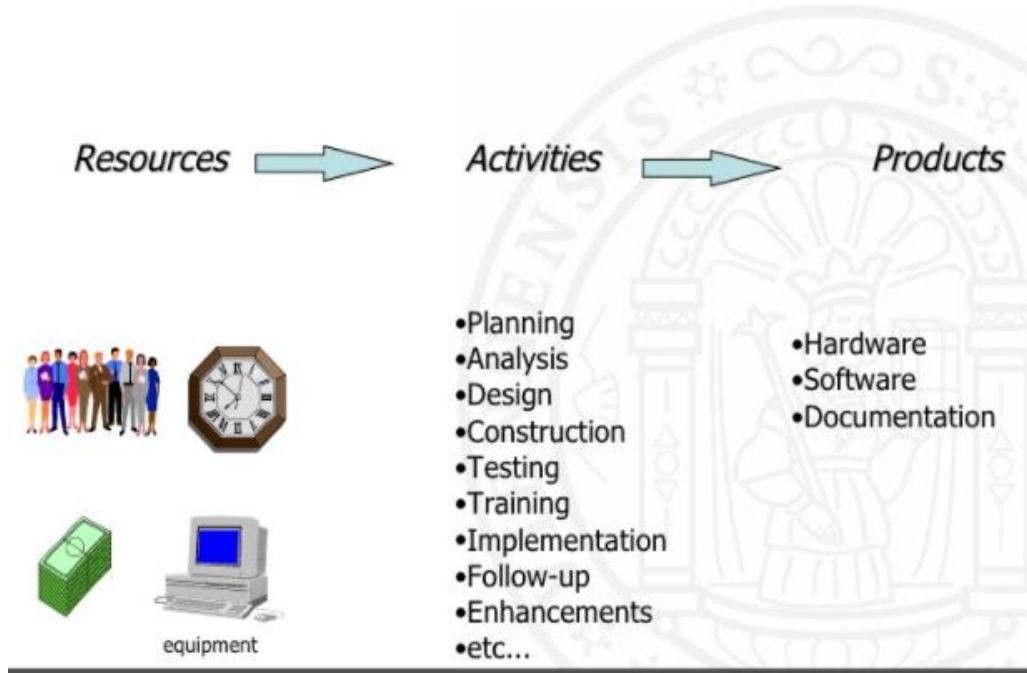
An existing organizational approach to software development can be assessed against those elements, and a meaningful strategy for improvement can be defined.

What is SPI?



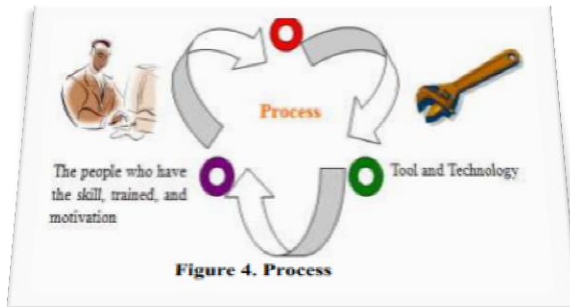
The SPI strategy transforms the existing approach to software development into something that is more focused, more repeatable, and more reliable (in terms of the quality of the product produced and the timeliness of delivery)

What is SPI?



Elements of Process

- A. People: The people who have the skills, training, and motivation.
- B. Rules and method: The rule and method to implement task.
- C. Tools and Technology: Techniques and tools must be needed.



BRAND CAMP

THE NEW PRODUCT WATERFALL



HOW DO WE
CHART OUR
ENTIRE COURSE
IF WE DON'T
KNOW WHAT'S
AHEAD?

PLAN



WHATEVER
HAPPENS, JUST
KEEP PADDLING!

BUILD



I WISH WE'D
DESIGNED FOR
THIS SCENARIO
UPFRONT

TEST



PATCH IT AS
BEST WE CAN.
NO TIME TO
CHANGE COURSE
NOW

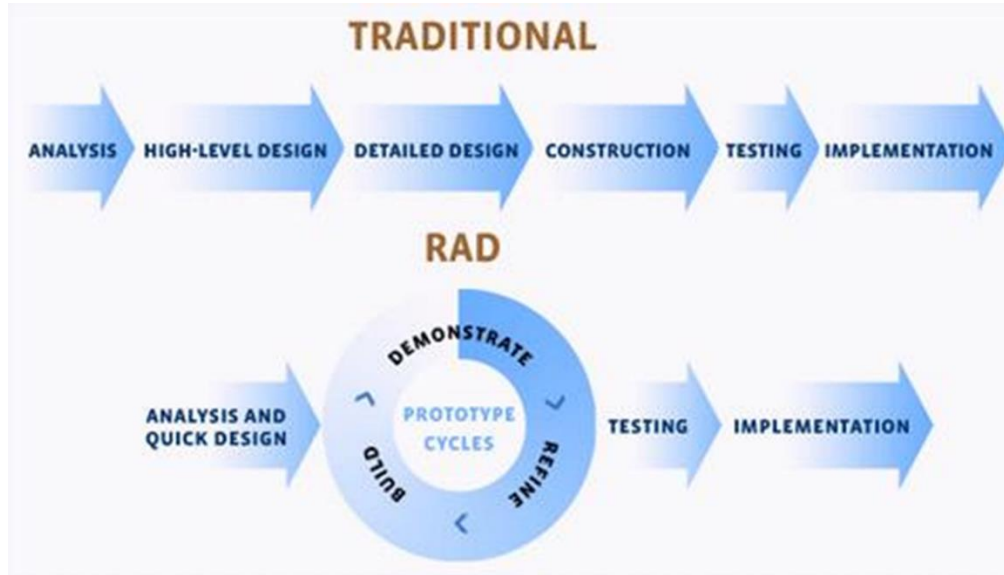
LAUNCH

by Tom Fishburne

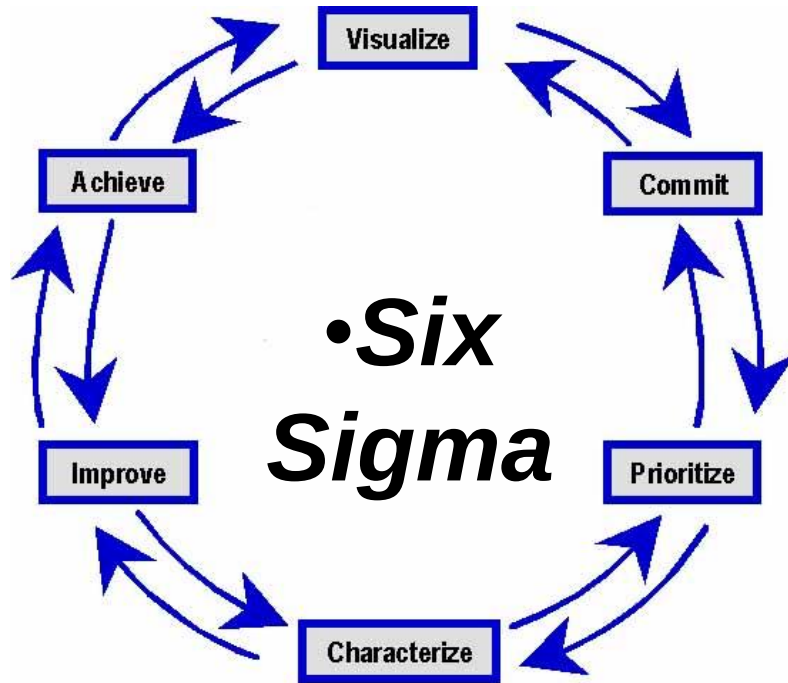
TOMFISHBURNE.COM

© 2010

RAD Approach



Quality Improvement – The Wheel of 6Sigma



Quality Improvement – The Wheel of 6Sigma

**Visualize – Understand how it works now and
imagine how it will work in the future**

**Commit – Obtain commitment to change from
the stakeholders**

**Prioritize – Define priorities for incremental
improvements**

Quality Improvement – The Wheel of 6Sigma

Characterize – Define existing process and define the time progression for incremental improvements

Improve – Design and implement identified improvements

Achieve – Realize the results of the change

Continuity and Independence of SQA

- Software Quality Assurance team must be independent in order to take an objective view of the process and report problems to senior management directly
- If prescribed process is inappropriate for the type of software product which is being developed, then it should be tailored

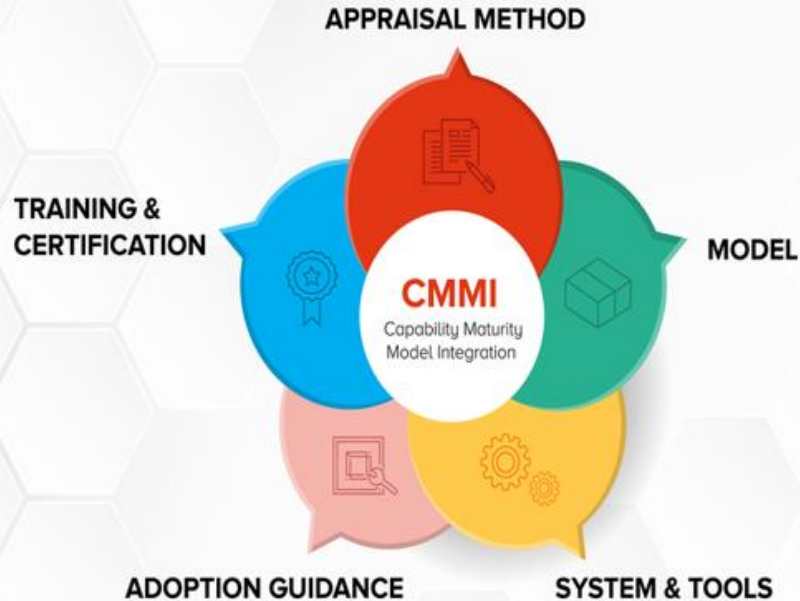
Continuity and Independence of SQA

- The standards must be upheld no matter how small the task. Prototyping doesn't mean no standards. It means tailored standards only.
- Quality is **FREE**, if it's **Everyone's Responsibility**

The CMMI Process Improvement Framework

- The CMMI framework is the current stage of work on process assessment and improvement that started at the Software Engineering Institute in the 1980s.
- The SEI's mission is to promote software technology transfer particularly to US defence contractors.
- It has had a profound influence on process improvement
 - ✓ Capability Maturity Model introduced in the early 1990s.
 - ✓ Revised maturity framework (CMMI) introduced in 2001.

Capability Maturity Model Integration



Capability Maturity Model Integration

The **Capability Maturity Model Integration** (CMMI) is a model that helps organizations to:
Effectuate process improvement.
Develop behaviors that decrease risks in service, product, and software development.

Capability Maturity Model Integration

CMMI's objectives for businesses include enabling your organization to:

Produce quality services or products

Improve customer satisfaction

Increase value for stockholders

Achieve industry-wide recognition for excellence

Grow market share

Process Capability Assessment

- Intended as a means to assess the extent to which an organization's processes follow best practice.
- By providing a means for assessment, it is possible to identify areas of weakness for process improvement.
- Ensure to follow various process assessment and improvement models.

The CMMI model

- An integrated capability model that includes software and systems engineering capability assessment.
- The model has two instantiations
 - Staged where the model is expressed in terms of capability levels;
 - Continuous where a capability rating is computed.

CMMI model components

Process areas

- 24 process areas that are relevant to process capability and improvement are identified. These are organized into 4 groups.

Goals

- Goals are descriptions of desirable organizational states. Each process area has associated goals.

Practices

- Practices are ways of achieving a goal - however, they are advisory and other approaches to achieve the goal may be used.

Process areas in the CMMI

Category	Process area
Process management	Organizational process definition (OPD)
	Organizational process focus (OPF)
	Organizational training (OT)
	Organizational process performance (OPP)
	Organizational innovation and deployment (OID)
Project management	Project planning (PP)
	Project monitoring and control (PMC)
	Supplier agreement management (SAM)
	Integrated project management (IPM)
	Risk management (RSKM)
	Quantitative project management (QPM)

Process areas in the CMMI

Category	Process area
Engineering	Requirements management (REQM)
	Requirements development (RD)
	Technical solution (TS)
	Product integration (PI)
	Verification (VER)
	Validation (VAL)
Support	Configuration management (CM)
	Process and product quality management (PPQA)
	Measurement and analysis (MA)
	Decision analysis and resolution (DAR)
	Causal analysis and resolution (CAR)

Goals and associated practices in the CMMI

Goal	Associated practices
The requirements are analyzed and validated, and a definition of the required functionality is developed.	Analyze derived requirements systematically to ensure that they are necessary and sufficient.
	Validate requirements to ensure that the resulting product will perform as intended in the user's environment, using multiple techniques as appropriate.
Root causes of defects and other problems are systematically determined.	Select the critical defects and other problems for analysis.
	Perform causal analysis of selected defects and other problems and propose actions to address them.
The process is institutionalized as a defined process.	Establish and maintain an organizational policy for planning and performing the requirements development process.

Examples of goals in the CMMI

Goal	Process area
Corrective actions are managed to closure when the project's performance or results deviate significantly from the plan.	Project monitoring and control (specific goal)
Actual performance and progress of the project are monitored against the project plan.	Project monitoring and control (specific goal)
The requirements are analyzed and validated, and a definition of the required functionality is developed.	Requirements development (specific goal)
Root causes of defects and other problems are systematically determined.	Causal analysis and resolution (specific goal)
The process is institutionalized as a defined process.	Generic goal

CMMI assessment

Examines the processes used in an organization and assesses their maturity in each process area.

Based on a 6-point scale:

Not performed

Performed

Managed

Defined

Quantitatively managed

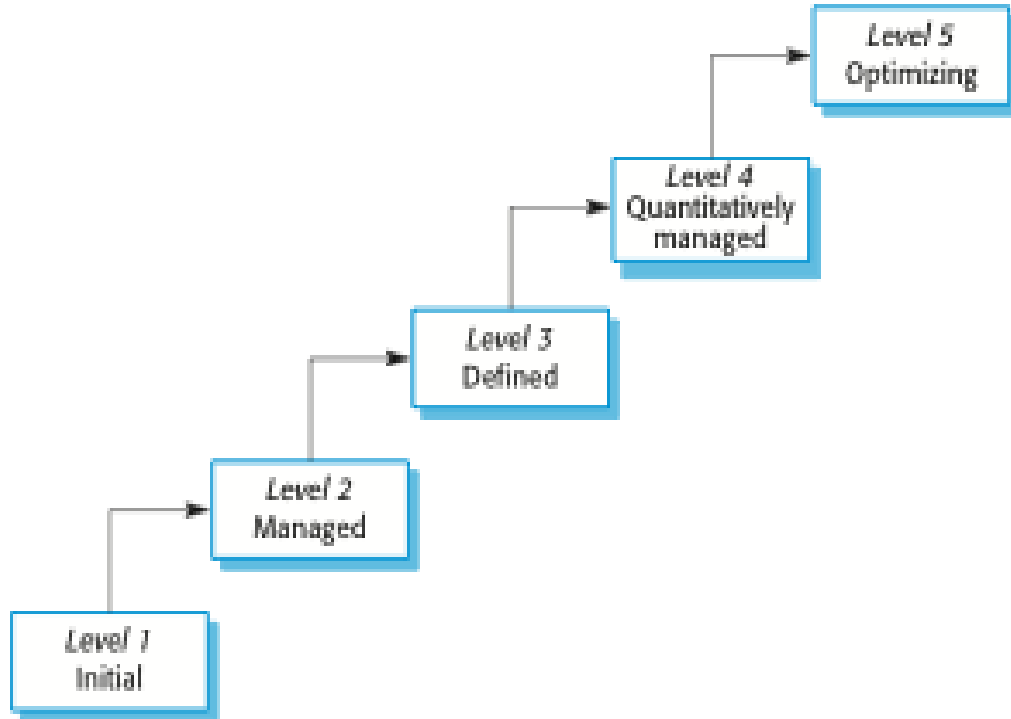
Optimizing

The staged CMMI model

- Comparable with the software CMM.
- Each maturity level has process areas and goals.
For example, the process area associated with the managed level include:

- Requirements management;
- Project planning;
- Project monitoring and control;
- Supplier agreement management;
- Measurement and analysis;
- Process and product quality assurance.

The CMMI staged maturity model



Flow of Process as a Project



Enjoy the Video

Learn Software Process Improvement

https://www.youtube.com/watch?v=_2GNHB_oYh4

Thank
You

Software Engineering

BLOCK 4: Advanced Topics in Software Engineering

MCS-213 Block-4 Unit-14 [Emerging Trends]

Topics to be Covered

Latest **2021 software development trends** that are sure to bring a paradigm shift in the IT industry



<https://www.sam-solutions.com/blog/software-development-trends/>

<https://www.forbes.com/sites/forbestechcouncil/2020/10/06/16-software-development-trends-that-will-soon-dominate-the-tech-industry/?sh=65d033144aa3>

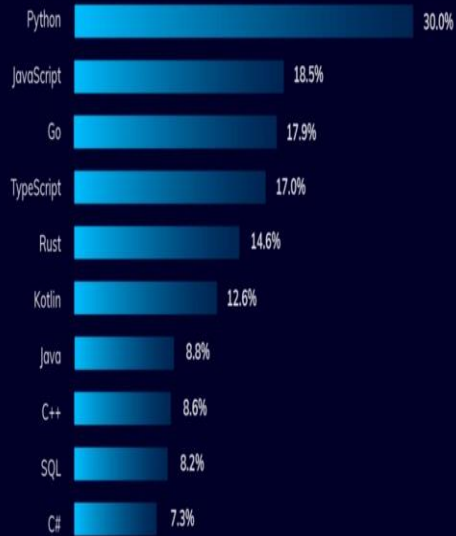
Top 7 **Software Development** **Trends** *of 2021*

Projected size of the IOT in healthcare market



Emerging trends

The most wanted top programming languages



Emerging Trends

<https://www.spaceo.ca/top-software-development-trends/>

1. What are the latest software engineering trends?

- Cloud-computing
- Low code, no code technology development
- Artificial intelligence & IoT
- Big data & Neuralink
- Mixed reality

2. Which is the least popular technology for software development?

No coding technology is currently the least popular for software development.

Emerging Trends

3. What is the future of the software industry?

The emergence of voice technology, time stamping, and a deeper need for automation suggests that the software engineering industry will continue to evolve for the next couple of years for business enhancements.

4. What is the least popular programming language in software development?

Objective- C, CoffeeScript, Perl, and Lua are some of the least popular and liked programming languages in software development methods.

Emerging Trends

- One of the things that I think we have learned is that we should all be very careful about making predictions about the future.” Bill Clinton.
- Software content in virtually every product and service will continue to grow—in some cases dramatically.

Emerging Trends

- Software must be demonstrably safe, secure, and reliable
- Requirements will emerge as systems evolve
- Interoperability and “networkability” will become dominant .

A “smart world” demands better, more reliable software

Understand Emerging Trends

Link : https://www.youtube.com/watch?v=0jR-kkls_LU

What is Reengineering?

- Michael Hammer and James Champy, entitled Reengineering the Corporation, as the important moment when reengineering became a movement.
- "Reengineering" functionally resembles planned change and what people have traditionally called "reorganization," but, in its beginnings, it came with a definite flavor of "starting from scratch," "blank slate," and "from the ground up."

What is Reengineering?

- **Software Re-engineering** is a process of software development which is done to improve the maintainability of a software system.
- Re-engineering is the examination and alteration of a system to reconstitute it in a new form.
- This process encompasses a combination of sub-processes like reverse engineering, forward engineering, reconstructing etc.

DEFINITION OF REENGINEERING

Reengineering is most commonly defined as the redesign of business processes—and the associated systems and organizational structures—to achieve a dramatic improvement in business performance.

DEFINITION OF *REENGINEERING*

- BPR [Business process reengineering] has been described as a radical new approach to business improvement, with the potential to achieve dramatic improvement in business performance.
- BPR should not be considered downsizing, restructuring, reorganization, and/or [new technology](#).

DEFINITION OF *REENGINEERING*

- 
- Rather it is the examination and change of five components of the business (1)strategy, (2)process, (3)technology, (4)organization, and (5)culture.
 - Many companies continue to experiment with reengineering, even if they have failed in previous attempts.
- 

MOTIVATION FOR REENGINEERING

The motivations for reengineering are many, including to:

Reduce costs/expenses (the most cited business-driven reengineering project goal)

Improve financial performance

Reduce external competition pressure

Reverse erosion of market share

Respond to emerging market opportunities

Improve customer satisfaction

Enhance quality of products and services

Difference

Qs. Is business process reengineering (BPR) same as business process improvement (BPI)?

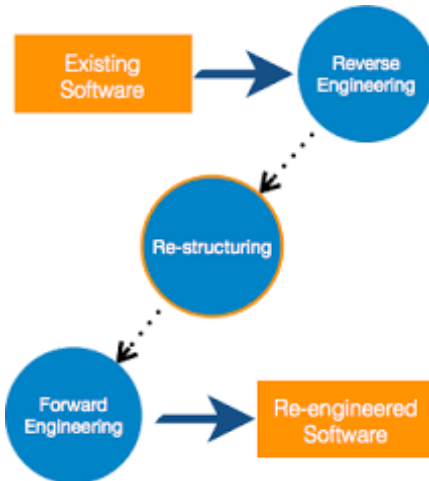
- On the surface, BPR sounds a lot like business process improvement (BPI). However, there are fundamental differences that distinguish the two. BPI might be about tweaking a few rules here and there.
- But reengineering is an unconstrained approach to look beyond the defined boundaries and bring in seismic changes.

Difference

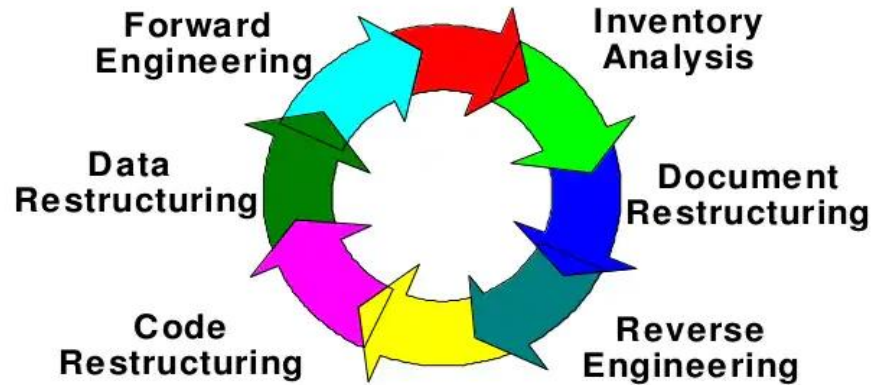
- While BPI is an incremental setup that focuses on tinkering with the existing processes to improve them, BPR looks at the broader picture. BPI doesn't go against the grain.
- It identifies the process bottlenecks and recommends changes in specific functionalities. The process framework principally remains the same when BPI is in play.
- BPR, on the other hand, rejects the existing rules and often takes an unconventional route to redo processes from a high-level management perspective.

Example

BPI is like upgrading the exhaust system on your project car. Business Process Reengineering, BPR, is about rethinking the entire way the exhaust is handled



Software Reengineering Process Model(II)



Difference

Difference between Reverse, Forward & Reengineering

- **Reverse Engineering**

It performs transformation from a large abstraction level to a higher one.

- **Forward Engineering**

It performs transformation from a higher abstraction level to a lower one.

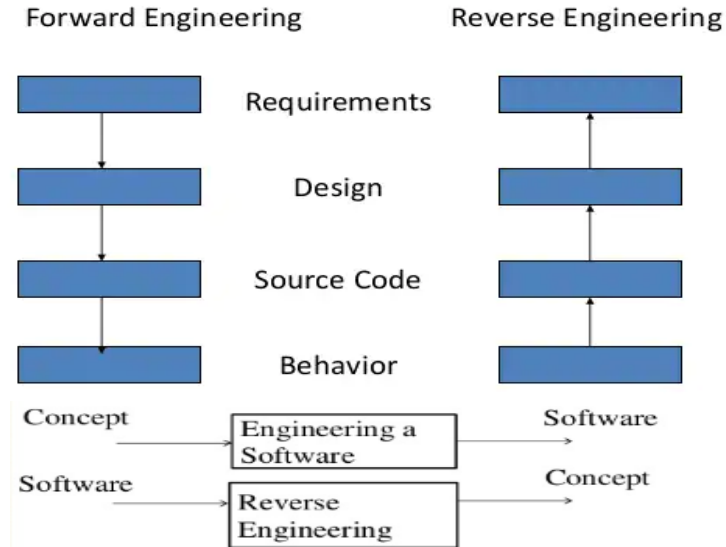
- **Reengineering**

It transforms an existing s/w system into a new but more maintainable system.

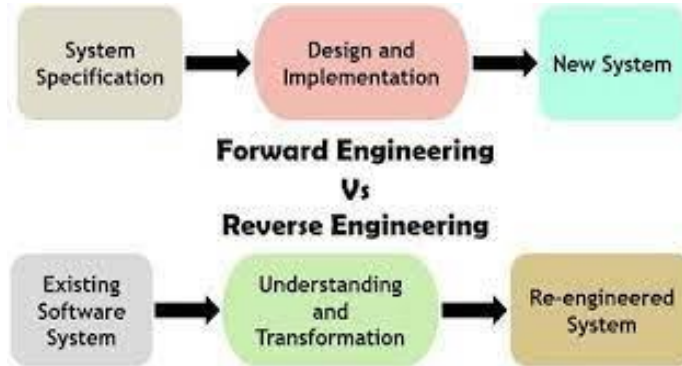
Reverse Engineering

Reverse engineering

Reverse engg. Vs Forward engg.



- In forward engineering, the **application are developed with the given** requirements. In reverse engineering or backward engineering, the information are collected from the given application. ... While Reverse Engineering or backward engineering takes less time to develop an application



When to Re-engineer

- When system changes are mostly confined to part of the system then re-engineer that part.
- When hardware or software support becomes obsolete.
- When tools to support re-structuring are available.

Resources

<https://www.encyclopedia.com/social-sciences-and-law/economics-business-and-labor/businesses-and-occupations/reengineering>

Thank You!

Software Engineering

BLOCK 4: Advanced Topics in Software Engineering

MCS-213

Block-4

Unit-15 [Introduction to UML]

What is UML?

- A standardized, graphical “modeling language” for communicating software design.
- Allows implementation-independent specification of:

user/system interactions (required behaviors)

partitioning of responsibility (OO)

integration with larger or existing systems

data flow and dependency

operation orderings (algorithms)

concurrent operations

What is UML?

Pretty pictures.

UML is not “process”. (That is, it doesn’t tell you how to do things, only what you should do.)

Uses for UML

- UML provides a sketch: to communicate aspects of system
 - ✓ forward design: doing UML before coding
 - ✓ backward design: doing UML after coding as documentation
- It act as a blueprint: a complete design to be implemented
 - ✓ sometimes done with CASE (Computer-Aided Software Engineering) tools

Motivations for UML

- UML is a fusion of ideas from several precursor modeling languages.
- We need a modeling language to:
 - ✓ help develop efficient, effective and correct designs, particularly Object Oriented designs.
 - ✓ communicate clearly with project stakeholders (concerned parties: developers, customer, etc.).
 - ✓ give us the “**big picture**” view of the project.

What is the use of UML?

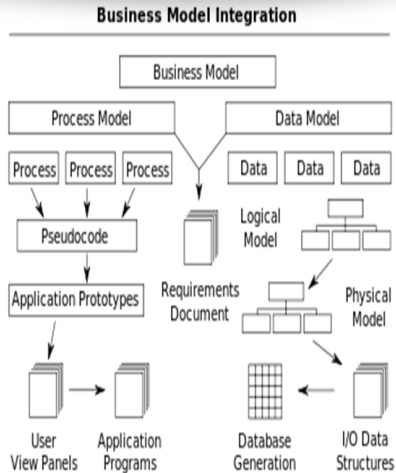
- UML has been used as a general-purpose modeling language in the field of software engineering. However, it has now found its way into the documentation of several [business processes](#) or [workflows](#).
- For example, activity diagrams, a type of UML diagram, can be used as a replacement for flowcharts. They provide both a more standardized way of modeling workflows as well as a wider range of features to improve readability and efficacy.

What is the use of UML?

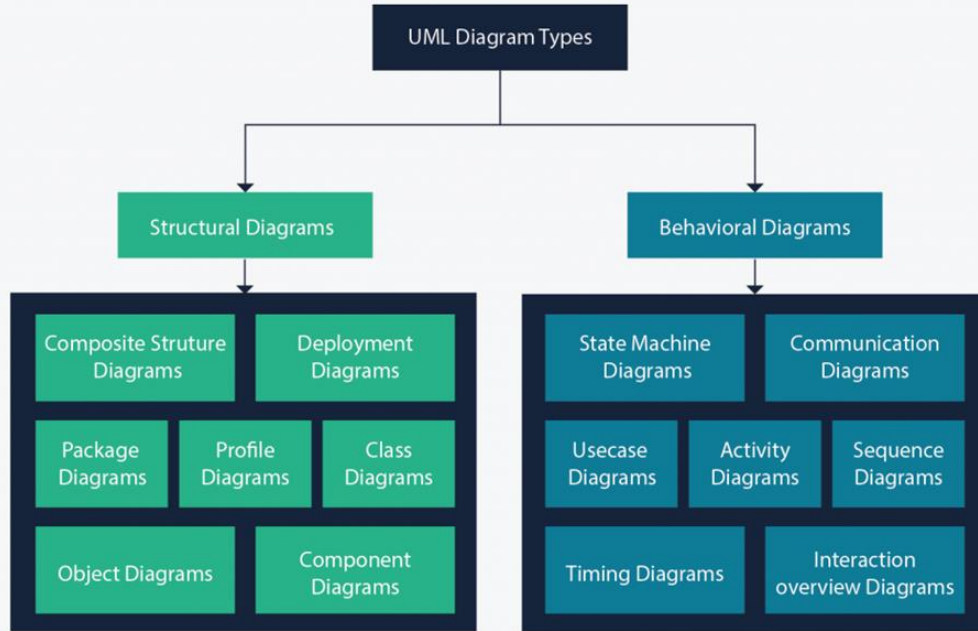
- UML diagrams are used to communicate different **aspects and characteristics** of a system. However, this is only a top-level view of the system and will most probably **not include** all the necessary details to execute the project until the very end.
- The UML diagram serves as a **complete design** that requires solely the actual operation of the system or software.
- Often, this is done by using **CASE** tools (Computer Aided Software Engineering Tools).

What is the use of UML?

- The main drawback of using CASE tools is that they require a certain level of expertise, user training as well as management and staff commitment.



Types of UML diagram



Structural UML diagrams

1. Class diagram

2. Package diagram

3. Object diagram

4. Component diagram

5. Composite structure diagram

6. Singleton

7. Profile diagram

Behavioral UML diagrams

Activity diagram

Sequence diagram

Use case diagram

State diagram

Communication diagram

Interaction overview diagram

Timing diagram

Note : All of the 14 different types of UML diagrams are not necessarily used for a project when documenting systems and/or architectures.

Popular use case diagrams

1. **Use case diagrams** : Describe the functional behavior of the system as seen by the user
2. **Class diagrams** : Describe the static structure of the system: Objects, attributes, associations
3. **Sequence diagrams** : Describe the dynamic behavior between objects of the system
4. **State chart diagrams** : Describe the dynamic behavior of an individual object
5. **Activity diagrams** : Describe the dynamic behavior of a system, in particular the workflow.

Types of Views

❖ Views that are considered when building an object-oriented software system:

1. Use Case view – This view exposes the requirements of a system.
2. Design View – Capturing the vocabulary.
3. Process View – modeling the distribution of the systems processes and threads.
4. Implementation view – addressing the physical implementation of the system.
5. Deployment view – focus on the modeling the components required for deploying the system



THANK YOU

Software Engineering

BLOCK 4: Advanced Topics in Software Engineering

MCS-213

Block-4

Unit-16 [Data Science for Software Engineers]

Topics to be Covered

What are CASE Tools

Categories of CASE Tools

Need of CASE Tools

Characteristics of a successful CASE Tool

CASE Tools and Requirement Engineering

Topics to be Covered

CASE Tools and Design and Implementation

Software Testing

Software Quality and CASE Tools

Software Project Management and CASE Tools

Profession

A Physician, a Civil Engineer and a Computer Scientist were arguing about what was the oldest profession in the world. The Physician remarked,

"Well, in the Bible, it says that God created Eve from a rib taken out of Adam. This clearly requires surgery, and so I can rightly claim that mine is the oldest profession in the world."

Profession

The **Civil Engineer** interrupted, and said,
" But even earlier in the book of Genesis, it states
that God created the order of the heavens and the
earth from out of the chaos. This was the first and
certainly the most spectacular application of civil
engineering. Therefore, fair doctor, you are wrong;
mine is the oldest profession in the world."

The **Computer Scientist** leaned back in the chair,
smiled and then said confidently,

"Ah, but what do you think created the chaos ? "

Scientist vs. Engineer

Computer Scientist:-

- Proves theorems about algorithms, designs languages, defines knowledge representation schemes
- Has infinite time...

Engineer:-

- Develops a solution for an application-specific problem for a client
- Uses computers & languages, tools, techniques and methods

Scientist vs. Engineer

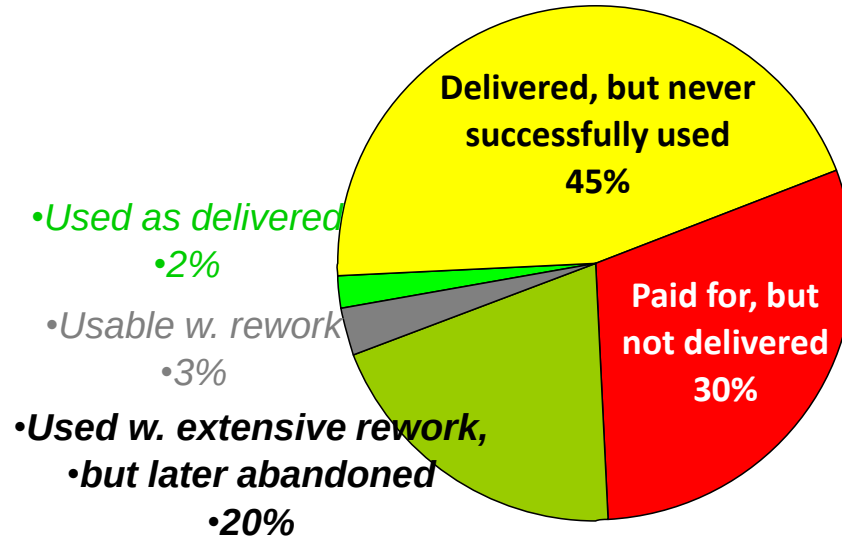
Software Engineer

- Works in multiple application domains
- Has only 3 months...
- while changes occurs in requirements and available technology

Why go for Data science in Software Engineering?

• *Take a look at the Report (The “Chaos” Report)*

9 software projects totaling \$96.7 million: Where The Money Went





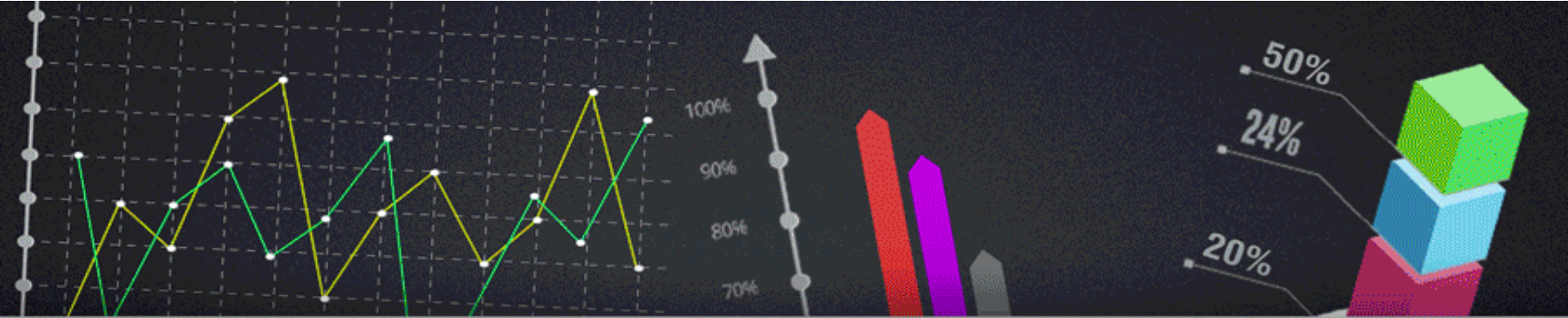
Data Science for Software Engineers

- ❖ Software engineering is a problem solving activity
 - Developing quality software for a complex problem within a limited time while things are changing
- ❖ There are many ways to deal with complexity
 - Modeling, decomposition, abstraction, hierarchy
 - Issue models: Show the negotiation aspects
 - System models: Show the technical aspects
 - Task models: Show the project management aspects
 - Use Patterns: Reduce complexity even further

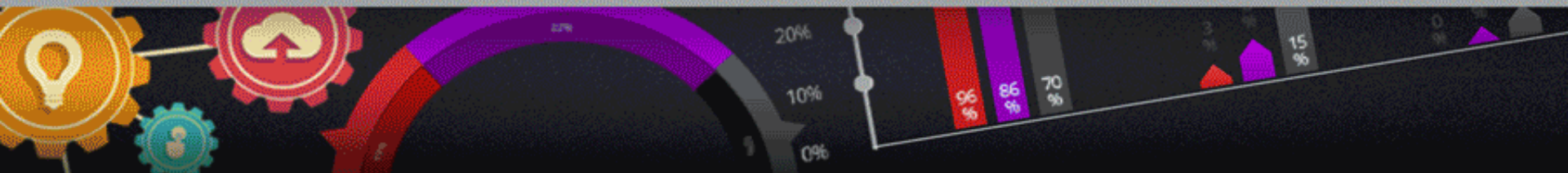
Data Science for Software Engineers

❖ Many ways to deal with change

- Tailor the software lifecycle to deal with changing project conditions
- Use a nonlinear software lifecycle to deal with changing requirements or changing technology
- Provide configuration management to deal with changing entities



WORLD *Needs* Data Scientist



Why should you think career as Data Scientist?

Data Scientist is the best job of the 21st century - Harvard Business Review

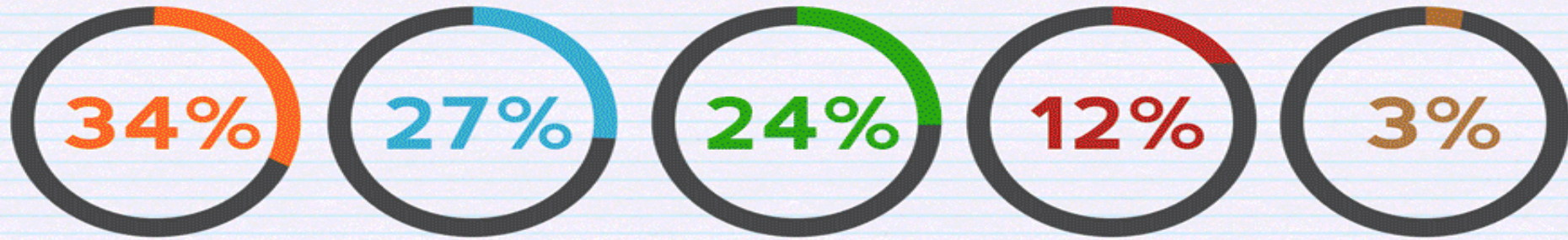
Global Big Data market to reach \$122B in revenue by 2025 – Frost & Sullivan

The US alone could face a shortage of 1.4 - 1.9 million Big Data Analysts by 2019 – Mckinsey

Why should you think career as Data Scientist?

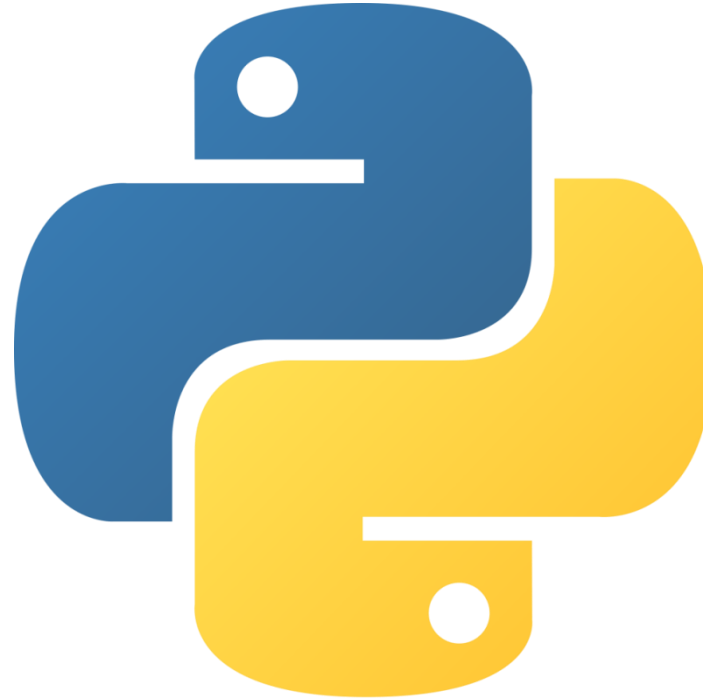
There is a serious shortage of Data Scientists and this is a major concern for Top MNCs around the world. All this means the major corporations are ready to pay top dollar salaries for professionals with the right Data Science skills.

Data Professionals were asked for **BEST SOURCE OF NEW DATA SCIENCE TALENT**



- Computer Science Students
- Professionals in fields other than I.T. or Computer Science
- Students in other Disciplines
- "Business Intelligence" Professionals
- Other

Why Python ???



What is Python?

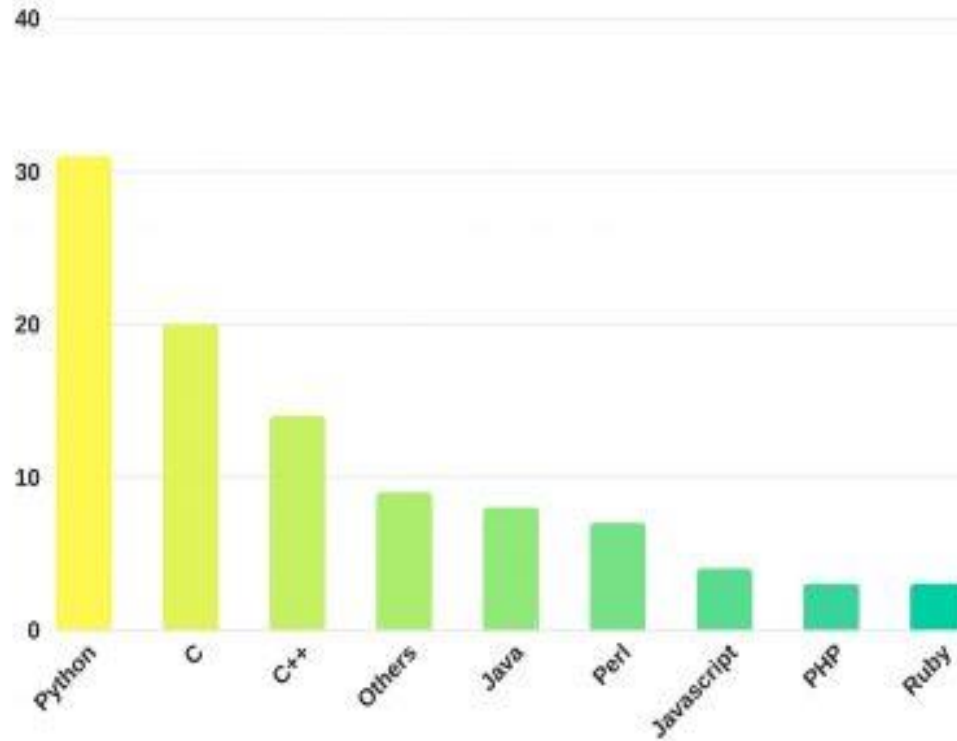
An all-purpose, general language that works on multiple platforms

High level and easy to learn.

More commonly used for **machine learning** and **predictive modeling** (particularly good for academics and data scientists)

Open source and free to learn and use more commonly by developers.





Source: <https://www.technotification.com/2018/04/python-best-programming-language-2018.html>

Why Is Python So Popular?

Java

```
public class Main { public static void  
main(String[] args) {  
System.out.println("hello world"); } }
```

Python

```
print('hello world')
```

Minimal setup is another of Python's perks.

Why Is Python So Popular?

The language continued to rank highly on various lists of the world's most popular programming languages.

Many programmers view Python as a language with a clean syntax and an expansive library.

Python's massive user base has created something of a positive feedback loop

In Python's case, it's Google, which uses the programming language in a number of applications (a corporate sponsor).

01

Readability

Python's syntax is very well thought out. Unlike many scripting languages (e.g. Perl), readability was a primary consideration when Python's syntax was designed.



02

Balance of High Level and Low Level Programming

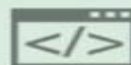
The ease of balancing high-level programming with low-level optimization is a particular strong point of Python code.



03

Language Interoperability

As a side affect of its universality, Python excels at gluing other languages together.



04

Documentation System

Brilliantly, Python incorporates module, class, function, and method documentation directly into the language itself.



05

Hierarchical Module System

Python uses modular programming, a popular system that naturally organizes functions and classes into hierarchical namespaces.



06

Data Structures

Good programming requires having and using the correct data structures for your algorithm.



07

Available Libraries

Python has an impressive standard library packaged with Python.



08

Testing Framework

Python provides a built-in, low barrier-to-entry testing framework that encourages good test coverage by making the fastest workflow, including debugging time, involve writing test cases.



What do businesses use python for?

❖ Building “data pipelines”:

- New data is coming in all the time
- Needs to be extracted, transformed and loaded
- Needs to be fast

❖ Descriptive Analytics

- These skills are in demand.
- Businesses want to know about their historical data.
- They also want to know what is happening right now.
- New marketing opportunities? Save time and money in current processes?

What do businesses use python for?

❖ Machine learning and data science?

- Can our customers be divided into clusters?
- Can we predict what a customer is likely to buy and make recommendations?
- Can we detect fraud? Can we predict risk?

Working as an analyst/scientist

- ❖ You may be familiar with some tools already, depending where you've come from:
 - Excel and Office tools
 - SPSS, MATLAB
 - SQL

Working as an analyst/scientist

- ❖ BI and analytics are a bit of a continuous process:
 - Cleaning data –missing values? Bad data?
 - Reshape data –is the data in the right format?
 - Loading –how much is there?
 - Find patterns –do these patterns add value?
 - Presentation –can you tell a story?

Working as an analyst/scientist





Thank You!

A graphic featuring the words "Thank You!" in a black, elegant cursive script. The text is centered and surrounded by several small, five-pointed yellow stars. A thick, horizontal yellow brushstroke is positioned beneath the word "You". The entire design is set against a plain white background.